

一个文件搞定。上半部分=编程题模板（抄+改）；下半部分=改错题手册（查毛病）。编程题按【编程目录】定位抄模板；改错题按【改错目录】定位查错因。

第一部分 编程题模板（抄了改）

编程目录

- 模板1 文件读取（逐行 / 边读边算 / 多数字一行 / Scanner / 混合内容 / 统计）
- 模板2 文件写入（逐行 / 列表 / 文本 / 追加 / BufferedWriter）
- 模板3 读+排序+写（组合题）
- 模板4 JDBC 数据库（连接/增删改查）
- 模板5 Swing GUI（5步骤 / Look&Feel / 三种加组件 / 五种布局 / 按钮弹窗）
- 模板6 事件处理（implements / 多监听器 / Adapter / 内部类匿名类）
- 模板7 多线程（Thread / Runnable）
- 模板8 常用小工具（异常、集合、字符串）

模板1 文件读取

1A 逐行读 + 转数字 + 排序输出（最经典）

```
import java.io.*;
import java.util.*;

public class ReadFile { // ★类名=文件名
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        try {
            BufferedReader br = new BufferedReader(
                new FileReader("D:/Sort.txt")); // ★文件路径
            String line;
            while ((line = br.readLine()) != null) { // 逐行读,读到null结束
                if (!line.trim().isEmpty()) {
                    list.add(Integer.parseInt(line.trim())); // 字符串转整数
                }
            }
            br.close();
            Collections.sort(list); // 升序排序
            // 降序: Collections.sort(list, Collections.reverseOrder());
            for (int num : list) { // 输出
                System.out.println(num);
            }
        } catch (IOException e) { // ★try必须配catch
            e.printStackTrace();
        }
    }
}
```

△ try 和 catch 必须成对！文件操作不写 try-catch 编译不过。

1B 边读边算（求和 / 最大 / 平均）

```
BufferedReader br = new BufferedReader(new FileReader("D:/in.txt"));
String line;
int sum = 0, count = 0, max = Integer.MIN_VALUE;
while ((line = br.readLine()) != null) {
    if (!line.trim().isEmpty()) {
        int n = Integer.parseInt(line.trim());
        sum += n; count++; // 累加、计数
        if (n > max) max = n; // 找最大
    }
}
br.close();
System.out.println("和="+sum+" 最大="+max+" 平均="+sum/count);
```

1C 一行有多个数字（空格/逗号分隔）

```
BufferedReader br = new BufferedReader(new FileReader("D:/in.txt"));
String line;
while ((line = br.readLine()) != null) {
    String[] parts = line.split(" "); // ★空格拆;逗号用 split(",")
    for (String p : parts) {
        if (!p.trim().isEmpty())
            System.out.println(Integer.parseInt(p.trim()));
    }
}
br.close();
```

1D 用 Scanner 读（直接读数字，不用 parseInt）

```
import java.util.Scanner;
import java.io.File;

try {
    Scanner sc = new Scanner(new File("D:/in.txt"));
```

```
while (sc.hasNextInt()) {           // 还有整数就继续
    int n = sc.nextInt();           // 直接读int
    System.out.println(n);
}
sc.close();
} catch (Exception e) {             // Scanner读文件catch这个
    e.printStackTrace();
}
// 读整行: while(sc.hasNextLine()){ String s = sc.nextLine(); }
```

注意：Scanner 读文件的异常是 FileNotFoundException，用 Exception 兜住最省事。

1E 混合内容（每行"姓名 年龄"这种，拆开存）

```
BufferedReader br = new BufferedReader(new FileReader("D:/data.txt"));
String line;
while ((line = br.readLine()) != null) {
    String[] parts = line.split(" ");           // 按空格拆
    String name = parts[0];                     // 第1个:姓名
    int age = Integer.parseInt(parts[1]);       // 第2个:年龄
    System.out.println(name + " 年龄 " + age);
}
br.close();
```

1F 统计行数 / 单词数

```
BufferedReader br = new BufferedReader(new FileReader("D:/in.txt"));
String line;
int lineCount = 0, wordCount = 0;
while ((line = br.readLine()) != null) {
    lineCount++;
    if (!line.trim().isEmpty())
        wordCount += line.split("\\s+").length; // 按空白拆算单词
}
br.close();
System.out.println("行数="+lineCount+" 单词数="+wordCount);
```

模板2 文件写入

2A 逐行写数字（最常考）

```
import java.io.*;
public class WriteNumbers {
    public static void main(String[] args) {
        int[] nums = {35, 12, 88, 7, 49};      // ★要写的数字
        try {
            PrintWriter pw = new PrintWriter(
                new FileWriter("D:/out.txt"));  // ★输出路径
            for (int n : nums) {
                pw.println(n);                  // 一行一个,println自动换行
            }
            pw.close();                         // 必须close,否则可能没写进去
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

△ 写数字用 println(n) 自动换行；千万别用 write(n)（会把数字当字符编码写，出乱码）。

2B 从 ArrayList 写

```
PrintWriter pw = new PrintWriter(new FileWriter("D:/out.txt"));
for (int n : list) pw.println(n);
pw.close();
```

2C 写文本字符串

```
PrintWriter pw = new PrintWriter(new FileWriter("D:/out.txt"));
pw.println("第一行");
pw.println("第二行");
pw.close();
```

2D 追加写（不覆盖，在末尾加）

```
// FileWriter 第二个参数 true = 追加;不写或false = 覆盖
PrintWriter pw = new PrintWriter(new FileWriter("D:/out.txt", true));
pw.println("追加的一行");
pw.close();
```

2E 用 BufferedWriter 写

```
BufferedWriter bw = new BufferedWriter(new FileWriter("D:/out.txt"));
for (int n : nums) {
    bw.write(String.valueOf(n)); // write要写字符串,数字先转String
    bw.newLine();               // 手动换行
}
bw.close();
```

模板3 读 + 排序 + 写（组合题，高频）

```
import java.io.*;
import java.util.*;

public class SortAndWrite {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        try {
            // 1.读
            BufferedReader br = new BufferedReader(new FileReader("D:/in.txt"));
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.trim().isEmpty())
                    list.add(Integer.parseInt(line.trim()));
            }
            br.close();
            // 2.排序
            Collections.sort(list);
            // 3.写
            PrintWriter pw = new PrintWriter(new FileWriter("D:/out.txt"));
            for (int n : list) pw.println(n);
            pw.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

模板4 JDBC 数据库（骨架死记）

六步：加载驱动 → 连接 → 建Statement → 执行SQL → 处理结果 → 关闭。

```
import java.sql.*;

public class TestDB {
    public static void main(String[] args) {
        try {
            // 1.加载驱动
            Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver"); // ★驱动(见表)
            // 2.连接: jdbc:类型://IP:端口;databaseName=库名
            String url = "jdbc:sqlserver://127.0.0.1:3130;databaseName=HUB"; // ★IP/端口/库
            Connection conn = DriverManager.getConnection(url, "Hust", "Hust"); // ★用户,密码
            Statement stmt = conn.createStatement();
            // 3.插入(增删改用 executeUpdate)
            String insert = "INSERT INTO Student(sCode, sName, sPenguin) " +
                "VALUES('20230001', '你的名字', '你的QQ')"; // ★表/字段/值
            stmt.executeUpdate(insert);
            // 4.查询(用 executeQuery 返回 ResultSet)
            ResultSet rs = stmt.executeQuery("SELECT * FROM Student"); // ★表名
            // 5.遍历输出
            while (rs.next()) {
                System.out.println(
                    rs.getString("sCode") + " " + // ★列名
                    rs.getString("sName") + " " +
                    rs.getString("sPenguin"));
            }
            // 6.关闭
            rs.close(); stmt.close(); conn.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

各数据库驱动名 + URL（替换第1、2步）：

数据库	驱动类名	URL
SQL Server	com.microsoft.sqlserver.jdbc.SQLServerDriver	jdbc:sqlserver://IP:端口;databaseName=库名
MySQL	com.mysql.cj.jdbc.Driver	jdbc:mysql://IP:端口/库名
Oracle	oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@IP:端口:库名

其它SQL（换掉第3步，都用 executeUpdate）： - 更新： UPDATE Student SET sName='新名' WHERE sCode='20230001' - 删除： DELETE FROM Student WHERE sCode='20230001'

死记骨架： Class.forName(驱动) → getConnection(url,用户,密码) → createStatement() → 增删改executeUpdate / 查executeQuery → while(rs.next()){rs.getString("列")} → close

模板5 Swing GUI（完全贴课件）

5-0 GUI 设计 5 步骤（课件核心）

- 引入 Swing 包 import javax.swing.*;
- 选 Look&Feel 外观（可选）
- 创建并设置窗口容器 JFrame
- 创建并添加组件
- 显示窗口 setVisible(true)

5-1 最简 GUI（HelloWorld）

```
import javax.swing.*;

public class HelloWorldSwing {
    public static void main(String[] args) {
```

```
JFrame frame = new JFrame("HelloWorldSwing");    // ★标题
JLabel label = new JLabel("Hello World!");        // ★标签内容
frame.add(label);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(200, 70);
frame.setVisible(true);

}

}
```

5-2 设置 Look&Feel（课件强调：要 try-catch）

```
try {
    UIManager.setLookAndFeel(
        "com.sun.java.swing.plaf.windows.WindowsLookAndFeel"); // Windows风格
    // 其它可选值：
    // UIManager.getCrossPlatformLookAndFeelClassName() // Java风格,不用try
    // UIManager.getSystemLookAndFeelClassName()        // 当前系统风格,不用try
    // "javax.swing.plaf.metal.MetalLookAndFeel"         // Java风格
} catch (Exception e) {
    e.printStackTrace();
}

// 也可在创建窗口前：JFrame.setDefaultLookAndFeelDecorated(true);
```

5-3 向 JFrame 加组件的三种方式（课件原文）

```
// 方式1：通过内容面板
frame.getContentPane().add(new JLabel("Hello!"));
// 方式2：建 JPanel 装组件,再setContentPane
JPanel contentPane = new JPanel();
JButton b = new JButton("确定");
contentPane.add(b);
frame.setContentPane(contentPane);
// 方式3(JDK1.5+,最常用)：直接add,自动转给内容面板
frame.add(b);
```

模板5补充 五种布局管理器（课件每种都考）

改布局只需改 setLayout(new XxxLayout(...)) 那一行。

A FlowLayout 流式（从左到右、自动换行；Panel默认）

```
setLayout(new FlowLayout());                // 居中(默认)
// setLayout(new FlowLayout(FlowLayout.LEFT, 10, 5)); // 左对齐,水平间距10,垂直5
add(new JButton("Button 1"));
add(new JButton("Button 2"));
```

对齐参数：FlowLayout.LEFT / CENTER / RIGHT（缺省居中）。

B BorderLayout 边界（5个区；Frame默认）

```
setLayout(new BorderLayout());
add(new JButton("North"), "North");          // 上
add(new JButton("South"), "South");          // 下
add(new JButton("East"), "East");            // 右
add(new JButton("West"), "West");            // 左
add(new JButton("Center"), "Center");        // 中
```

五个区：North/South/East/West/Center。

C GridLayout 网格（n行m列,等大;左到右上到下填充）

```
setLayout(new GridLayout(3, 2));            // 3行2列
add(new JButton("1")); add(new JButton("2"));
add(new JButton("3")); add(new JButton("4"));
add(new JButton("5")); add(new JButton("6"));
// 行优先填充;rows或cols可填0表示任意数目;先满足行
```

D CardLayout 卡片（叠放,一次显示一张,可切换）

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class CardDemo extends JFrame implements ActionListener {
    JPanel cards;
    CardLayout cl = new CardLayout();
    public CardDemo() {
        setLayout(new BorderLayout());
        JPanel cp = new JPanel();
        JButton bt = new JButton("卡片切换");
        bt.addActionListener(this);    // 按钮注册监听
        cp.add(bt);
        add("North", cp);
        cards = new JPanel();
        cards.setLayout(cl);            // cards用CardLayout
        JPanel p1 = new JPanel(); p1.add(new JButton("Button 1"));
        JPanel p2 = new JPanel(); p2.add(new JTextField("TextField", 20));
        cards.add("卡片1", p1);
        cards.add("卡片2", p2);
        add("Center", cards);
    }
}
```

```
public void actionPerformed(ActionEvent e) {
    cl.next(cards);           // 显示下一张卡片
    // 其它: cl.first/previous/last/show(cards,"名字")
}
public static void main(String[] args) {
    CardDemo w = new CardDemo();
    w.setSize(300,200);
    w.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    w.setVisible(true);
}
}
```

E GridBagLayout / BoxLayout（课件只提名，了解即可）

- GridBagLayout：网格包，组件可不等大、可跨行跨列。
- BoxLayout：箱式，组件排成一行或一列（Swing新增）。

选哪个布局（课件原文）：

- 一行紧凑显示 → FlowLayout
- 大小相同成行成列 → GridLayout
- 充满容器空间 → BorderLayout 或 GridBagLayout

模板5补充2 按钮点击 + 弹窗（综合，应对GUI编程题）

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MyGUI extends JFrame {
    public MyGUI() {
        setTitle("窗口标题");           // ★标题
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);                 // 绝对布局,手动摆位置
        String[] names = {"楼A", "楼B", "楼C"};           // ★按钮/楼名
        int[][] pos = {{50,50},{200,50},{50,150}};       // ★每个x,y
        for (int i = 0; i < names.length; i++) {
            JButton btn = new JButton(names[i]);
            btn.setBounds(pos[i][0], pos[i][1], 120, 60); // x,y,宽,高
            add(btn);
            btn.addActionListener(new ActionListener() { // 匿名类监听
                public void actionPerformed(ActionEvent e) {
                    JOptionPane.showMessageDialog(null, "表拦我！我要复学！！"); // ★弹窗文字
                }
            });
        }
        setVisible(true);
    }
    public static void main(String[] args) {
        new MyGUI();
    }
}
```

弹窗：JOptionPane.showMessageDialog(null, "内容"); 课件其它对话框：showInputDialog(输入)、showConfirmDialog(确认)、showOptionDialog(选项)。用 setBounds 手动摆位置时必须 setLayout(null)。

模板6 事件处理（课件重头戏，4种写法）

事件模型3要素：事件(Event)、事件源(组件)、监听器(Listener)。**机制**：组件注册监听器 addXxxListener(监听器) → 事件发生 → 自动调监听器的方法。

写法1 让类自己实现监听器接口（implements）

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class TestButton {
    public static void main(String[] args) {
        JFrame f = new JFrame("Test");
        f.setSize(200, 100);
        f.setLayout(new FlowLayout());
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JButton b = new JButton("Press Me!");
        b.addActionListener(new ButtonHandler()); // 注册监听器
        f.add(b);
        f.setVisible(true);
    }
}
// 单独的监听器类
class ButtonHandler implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("被点击了");
        System.out.println("按钮文字:" + e.getActionCommand());
    }
}
```

写法2 多监听器 / 一个类实现多个监听接口

```
// 一个类同时实现多个监听接口
public class TwoListener implements MouseMotionListener, MouseListener {
    JFrame f;
    JTextField tf;
```

```
public void go() {
    f = new JFrame("Two Listener");
    tf = new JTextField(30);
    f.add(tf, BorderLayout.SOUTH);
    f.addMouseMotionListener(this);    // 注册鼠标移动监听
    f.addMouseListener(this);        // 注册鼠标点击监听
    f.setSize(300,200);
    f.setVisible(true);
}
// MouseMotionListener 的方法(2个)
public void mouseDragged(MouseEvent e) {}
public void mouseMoved(MouseEvent e) {}
// MouseListener 的方法(5个,必须全写)
public void mouseEntered(MouseEvent e) { tf.setText("鼠标进入"); }
public void mouseClicked(MouseEvent e) {}
public void mouseReleased(MouseEvent e) {}
public void mousePressed(MouseEvent e) {}
public void mouseExited(MouseEvent e) {}
public static void main(String[] args) { new TwoListener().go(); }
}
```

implements 监听接口必须实现它的**所有**方法,一个不能少（这是改错题考点）。

写法3 适配器 Adapter（只重写需要的方法）

```
// Adapter已把接口所有方法空实现,继承它只重写的
public class MouseClickHandler extends MouseAdapter {
    public void mouseClicked(MouseEvent e) {
        System.out.println("点击坐标:" + e.getX() + "," + e.getY());
    }
    // 其它方法不用写,继承MouseAdapter的空实现
}
```

课件原文：用户定义的监听器只能继承一个适配器类，且继承后不能再继承其它类。

写法4 内部类 / 匿名类（最常用，避免多继承限制）

```
// 匿名类:当场写一个没名字的监听器
public class MyGUI {
    public MyGUI() {
        JButton btn = new JButton("按钮");
        btn.addMouseListener(new MouseAdapter() { // 匿名类
            public void mouseClicked(MouseEvent e) {
                System.out.println("被点击");
            }
        });
    }
    // 结尾 });
}
```

内部类方式：在类里定义内部类继承Adapter，既用Adapter又避开"主类已继承JFrame不能再继承"的限制。

常见事件、监听接口、方法对照：

事件	监听接口	主要方法
按钮点击 ActionEvent	ActionListener	actionPerformed
鼠标点击 MouseEvent	MouseListener	mouseClicked/Pressed/Released/Entered/Exited(5个)
鼠标移动 MouseEvent	MouseMotionListener	mouseDragged/mouseMoved(2个)
键盘 KeyEvent	KeyListener	keyPressed/keyReleased/keyTyped(3个)
窗口 WindowEvent	WindowListener	windowClosing等(7个)
选项 ItemEvent	ItemListener	itemStateChanged

对应的 **Adapter（方法多的接口才有）**：MouseAdapter、KeyAdapter、WindowAdapter、MouseMotionAdapter。（ActionListener 只1个方法，没有 Adapter。）

模板7 Swing 画图（Graphics 绘图）

关键：要点击的→用按钮；纯画的图形(线/矩形/形状)→重写 paintComponent 画。

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class DrawDemo extends JFrame {
    public DrawDemo() {
        setTitle("画图窗口");
        setSize(500, 350);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel panel = new JPanel() {
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);    // 必须第一行(清背景)
                g.setColor(Color.GRAY);
                g.fillRect(0, 150, 500, 25);    // 实心矩形(如桥面): x,y,宽,高
                g.drawLine(0, 150, 500, 150);    // 线段: x1,y1,x2,y2
                g.drawRect(60, 175, 12, 40);    // 空心矩形(如桥墩)
                g.setColor(Color.BLACK);
                g.drawString("高架桥", 230, 145); // 写字: 内容,x,y
            }
        };
        panel.setLayout(null);
        String[] names = {"东九楼", "东一楼"};    // ★楼名(按钮)
```

```
int[][] pos = {{60,40},{300,40}};
for (int i = 0; i < names.length; i++) {
    JButton btn = new JButton(names[i]);
    btn.setBounds(pos[i][0], pos[i][1], 130, 70);
    panel.add(btn);
    btn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            JOptionPane.showMessageDialog(null, "表拦我! 我要复学! ! "); // ★弹窗文字
        }
    });
}
add(panel);
setVisible(true);
}

public static void main(String[] args) {
    new DrawDemo();
}
}
```

Graphics 画图方法： - g.setColor(Color.RED) 设颜色(RED/BLUE/GRAY/BLACK/GREEN/YELLOW/WHITE) - g.fillRect(x,y,宽,高) 实心矩形 / g.drawRect(x,y,宽,高) 空心矩形 - g.drawLine(x1,y1,x2,y2) 线段 - g.fillOval(x,y,宽,高) 实心椭圆 / g.drawOval(...) 空心椭圆 - g.fillPolygon(int[] x, int[] y, int n) 实心多边形 / g.drawPolygon(...) 空心 - g.drawArc(x,y,宽,高,起始角,弧度) 圆弧 - g.drawString("字",x,y) 写文字 - g.setFont(new Font("宋体", Font.BOLD, 20)) 设字体

画图什么时候刷新：动态改了内容后调 repaint() 触发重绘。

模板8 多线程

写法A：继承 Thread

```
public class MyThread extends Thread {
    private String name;
    public MyThread(String name) { this.name = name; }
    public void run() { // ★重写run
        for (int i = 0; i < 5; i++)
            System.out.println(name + ": " + i);
    }
    public static void main(String[] args) {
        new MyThread("线程A").start(); // 用start()不是run()!
        new MyThread("线程B").start();
    }
}
```

写法B：实现 Runnable

```
public class MyRunnable implements Runnable {
    private String name;
    public MyRunnable(String name) { this.name = name; }
    public void run() { // ★实现run
        for (int i = 0; i < 5; i++)
            System.out.println(name + ": " + i);
    }
    public static void main(String[] args) {
        new Thread(new MyRunnable("线程A")).start();
        new Thread(new MyRunnable("线程B")).start();
    }
}
```

△启动用 start() 不是 run()! run()只是普通方法调用，不开新线程（改错题常考）。

模板8 常用小工具

集合 ArrayList:

```
ArrayList<Integer> list = new ArrayList<>();
list.add(5); // 加
list.get(0); // 取第0个
list.size(); // 个数
list.remove(0); // 删第0个
Collections.sort(list); // 升序
Collections.sort(list, Collections.reverseOrder()); // 降序
Collections.max(list); // 最大
Collections.min(list); // 最小
```

字符串常用：

```
s.length(); // 长度
s.charAt(0); // 第0个字符
s.substring(1,3); // 截取[1,3)
s.split(" "); // 按空格拆成数组
s.equals(t); // 比内容(不能用==)
s.trim(); // 去首尾空格
s.toUpperCase()/toLowerCase();// 大小写
Integer.parseInt(s); // 字符串转int
String.valueOf(n); // int转字符串
```

异常处理结构：

```
try {
    // 可能出错的代码
} catch (异常类型 e) {
```

```
e.printStackTrace();    // 打印错误
} finally {
    // 不管出不出错都执行(可选)
}
```

拿到编程题"改哪里"对照表

模板	改这几处
1 读文件	文件路径、转换(parseInt/不转)、处理(排序/求和/统计)
2 写文件	输出路径、写的内容、覆盖还是追加(true)
3 读读写	输入路径、输出路径
4 JDBC	驱动名、IP/端口/库名、用户密码、表名、字段、SQL
5 GUI按钮	标题、按钮名、位置、弹窗文字
6 画图	画什么(fillRect/drawLine)、楼名、弹窗文字
7 多线程	线程名、run()里做什么

第二部分 改错题手册（查毛病）

改错目录

A 类与继承: A1顶层类private／A2重写缩小权限／A3final类被继承／A4final方法被重写／A5多个public类／A6类名≠文件名／A7构造没调super **B 抽象abstract**: B1抽象方法带{}／B2有抽象方法类没abstract／B3abstract+final／B4abstract+private/static／B5new抽象类／B6子类没实现全部抽象方法 **C 接口interface**: C1实现方法不是public／C2接口方法带方法体／C3改接口字段／C4没实现全部／C5extends/implements搞反／C6接口成员加private **D final**: D1变量二次赋值／D2final引用改内容(陷阱)／D3构造方法final／D4lambda改外部变量 **E static**: E1static方法访问非static／E2static用this／E3非static内部类有static／E4static当重写 **F 内部类**: F1直接new成员内部类／F2匿名类语法／F3匿名类改局部变量 **G 重写多态**: G1重写签名不符／G2static当重写／G3字段无多态 **H 常见语法**: H1==比字符串／H2数组越界/length／H3包装类==／H4switch漏break／H5构造方法写返回类型／H6start用成run

A. 类与继承

A1 顶层类用 private/protected

```
private class Foo {}    // ✖
```

错因：顶层类只能 public 或 default(不写)。改：去掉private→class Foo。（注意：内部类可以用四种权限，这条只针对顶层类。）

A2 重写方法缩小权限

```
class P { public void f(){} }
class C extends P { void f(){} }    // ✖ public缩成default
```

错因：重写权限只能放大或不变，不能缩小(破坏多态)。改：public void f(){}。权限大小：private < default < protected < public。

A3 final 类被继承

```
final class A{}  class B extends A{}    // ✖
```

改：去掉A的final，或B别继承。

A4 final 方法被重写

```
class P{ final void f(){} }
class C extends P{ void f(){} }    // ✖
```

改：去掉父类方法的final。

A5 一个文件多个 public 类

```
public class A{}  public class B{}    // ✖
```

错因：一个.java最多一个public类。改：只留一个public，其余去掉。

A6 public 类名≠文件名 错因：public类名必须和文件名完全一致(含大小写)。改：改类名或文件名使一致。

A7 子类构造没正确调 super

```
class P{ P(int x){} }    // 父类只有带参构造
class C extends P{ C(){} }    // ✖ 默认super()但父类没无参构造
```

改：C构造里第一行写 super(参数)；（super必须是第一句）。

B. 抽象 abstract

B1 抽象方法带方法体(含空{})

```
abstract void f() {}    // ✖ 空{}也不行
abstract void f();    // ✔
```

改：去掉{}用分号。

B2 有抽象方法但类没abstract

```
class A{ abstract void f(); }    // ✖
```

改：abstract class A。

B3 abstract + final

```
final abstract class A{}    // ✖ 矛盾
```

改：去掉一个(通常去final)。

B4 abstract + private/static

```
private abstract void f();    // ✖
```



```
static abstract void f(); // ✖
```

错因: abstract要被重写, private看不见, static不重写, 都矛盾。改: 去掉private/static。

B5 直接 new 抽象类

```
abstract class A{ A a=new A(); // ✖
```

改: 用子类 new 子类() 或匿名类 new A(){实现方法}。

B6 子类没实现全部抽象方法又没声明abstract

```
abstract class A{ abstract void f(); abstract void g(); }
class B extends A{ void f(){} } // ✖ 缺g()
```

改: 实现g(), 或B也声明abstract。

C. 接口 interface

C1 实现接口方法不是public

```
interface I{ void f(); }
class C implements I{ void f(){} } // ✖ 默认权限=缩小
```

改: public void f(){}。

C2 接口方法带方法体(传统)

```
interface I{ void f(){} } // ✖
```

改: void f();。要带实现用 default void f(){}。

C3 修改接口字段

```
interface I{ int X=10; } I.X=20; // ✖ X是public static final常量
```

改: 不能改。

C4 implements多个没实现全部

```
class C implements A,B{ } // ✖ A、B方法都要实现
```

改: 所有方法都实现(都加public)。

C5 extends/implements搞反

```
class C extends MyInterface{ } // ✖ 接口用implements
class C implements MyClass{ } // ✖ 类用extends
```

规则: 类实现接口→implements; 类继承类→extends; 接口继承接口→extends。

C6 接口成员加private/protected

```
interface I{ protected void f(); } // ✖
```

改: 接口方法只能public(默认), 去掉修饰符。

D. final

D1 final变量二次赋值

```
final int x=5; x=6; // ✖
```

改: 只能赋值一次。

D2 final引用改内容(陷阱:其实合法)

```
final int[] arr={1,2,3};
arr[0]=99; // ✔ 合法!改内容
arr=new int[5]; // ✖ 改引用才错
```

判断: 改 arr[i]/对象内容→不算错; 改 arr=新对象→错。

D3 构造方法用final

```
final Foo(){ } // ✖ 构造方法不能final(也不能abstract/static)
```

改: 去掉final。

D4 lambda/匿名类改外部局部变量

```
int c=0; Runnable r=()->{ c++; }; // ✖
```

改: 只能读不能改; 要累加用数组或成员变量绕开。

E. static

E1 static方法访问非static成员

```
class A{ int x; static void f(){ System.out.println(x); } } // ✖
```

错因: static属于类、没this, 访问不了实例成员。改: x改static, 或f()改非static, 或方法内new对象访问。

E2 static方法用this

```
static void f(){ this.x=1; } // ✖ static无this
```

改: 去掉this。

E3 非static内部类有static成员(旧版本)

```
class Outer{ class Inner{ static int x; } } // ✖ Java16前
```

改: Inner改static class, 或x去掉static。(Java16+已放开)

E4 static方法当重写

```
class P{ static void f(){} }
class C extends P{ @Override static void f(){} } // ✖
```

错因：static是隐藏不是重写，不能@Override。改：去掉@Override。

F. 内部类

F1 直接new成员内部类(没外部对象)

```
Outer.Inner i=new Outer.Inner(); // ✖
```

改：

```
Outer o=new Outer();
Outer.Inner i=o.new Inner(); // ✔
```

(若Inner是static内部类，new Outer.Inner() 才对。)

F2 匿名类/适配器语法

```
btn.addMouseListener(new MouseAdapter { // ✖ 少()
    public void mouseClicked(MouseEvent e){
    }
});
```

改：new MouseAdapter() 带括号，结尾});。

F3 匿名类改局部变量

```
int n=5;
new Runnable(){ public void run(){ n=9; } }; // ✖
```

改：只读不改。

G. 重写 vs 重载 / 多态

G1 重写签名不符

```
class P{ void f(int x){} }
class C extends P{ void f(String x){} } // 这是重载不是重写
```

要点：重写要名字+参数+返回值一致；参数变了=重载。改：参数与父类一致。

G2 static当重写：见E4，static同名是隐藏。

G3 字段无多态

```
class P{ int x=1; } class C extends P{ int x=2; }
P p=new C(); System.out.println(p.x); // 输出1不是2!
```

要点：方法看实际对象类型(多态)；字段看引用声明类型(无多态)。p.x按P算=1。

H. 常见语法

H1 == 比字符串内容

```
if(s1==s2) // ✖ 比地址
```

改：s1.equals(s2)。

H2 数组越界/length

```
int[] a=new int[5]; a[5]=1; // ✖ 下标0~4
a.length() // ✖ 数组是length属性,无括号
```

改：下标0~length-1；数组用a.length(无括号)，字符串才是s.length()。

H3 包装类 ==

```
Integer a=128,b=128; if(a==b) // ✖ 超出-128~127缓存,==为false
```

改：用.equals() 或 a.intValue()==b.intValue()。

H4 switch漏break

```
switch(x){ case 1: doA(); case 2: doB(); } // ✖ 没break会贯穿
```

改：每个case末尾加break；。

H5 构造方法写返回类型

```
class A{ void A(){} } // ✖ 这是普通方法不是构造!
```

错因：构造方法不能有返回类型(连void都不行)。改：去掉void→A(){}。

H6 启动线程用了run

```
new MyThread().run(); // ✖ run只是普通调用,不开新线程
```

改：new MyThread().start();。

改错题 30 秒排查顺序

- 1. 看权限：顶层类有没private/protected(A1)；重写有没缩小(A2/C1)
- 2. 看abstract：抽象方法带没带{}(B1)；该不该abstract(B2)；冲不冲突(B3/B4)
- 3. 看final：变量改没改(D1)；引用vs内容(D2陷阱)
- 4. 看static：碰没碰实例成员/this(E1/E2)
- 5. 看接口：implements方法public没(C1)；extends/implements反没(C5)
- 6. 看构造：写没写返回类型(H5)；super对不对(A7)
- 7. 看==：字符串/包装类用没用==(H1/H3)

8. 看new: 成员内部类给外部对象没(F1); 抽象类被new没(B5)
9. 看线程: start写成run没(H6)
10. 看多态: 字段按引用类型(G3); switch漏break(H4)

编程抄第一部分，改错查第二部分，开卷通杀。

Java 基本数据类型与 String 互相转换完整总结

一、int 类型与 String 转换（计算器题目专用前置总结）

1. int → String（给文本框 setText 场景） 设 int num = 123; 字符串拼接（作业最简写法） java 运行 String s1 = num + ""; String.valueOf() 标准通用写法 java 运行 String s2 = String.valueOf(num); Integer.toString() 专用静态方法 java 运行 String s3 = Integer.toString(num); 自动装箱后调用 toString()（不可直接 num.toString()） java 运行 Integer obj = num; // 自动装箱 String s4 = obj.toString();
2. String → int（读取文本框 getText 场景） 设 String str = "123"; Integer.parseInt()（最常用，返回基础 int） java 运行 int n1 = Integer.parseInt(str); Integer.valueOf()（返回 Integer 对象，自动拆箱） java 运行 Integer n2 = Integer.valueOf(str); int num = n2; 注意： 字符串非数字会抛出 NumberFormatException
3. 易错点 int 是基础数据类型，不存在 .toString() 方法，直接写 res.toString() 编译报错； 只有装箱后的 Integer 对象才能调用 .toString()。## 二、Java 8 种全部基本数据类型 ↔ String 转换大全 8 种基础类型: byte short int long float double char boolean （一）所有基本类型 → String 方案 1：字符串拼接（作业首选，最简单） 任意基础类型拼接空字符串，自动转为字符串 java 运行

byte b = 1; short s = 2; int i = 3; long l = 4L; float f = 5.5F; double d = 6.6; char c = 'A'; boolean bl = true;

String sb = b + ""; String ss = s + ""; String si = i + ""; String sl = l + ""; String sf = f + ""; String sd = d + ""; String sc = c + ""; String sbl = bl + ""; 方案 2：String.valueOf(变量)（项目标准通用写法） 统一 API，支持全部 8 种基本类型与对象 java 运行 String sb = String.valueOf(b); String ss = String.valueOf(s); String si = String.valueOf(i); String sl = String.valueOf(l); String sf = String.valueOf(f); String sd = String.valueOf(d); String sc = String.valueOf(c); String sbl = String.valueOf(bl); 方案 3：对应包装类静态方法 Xxx.toString(值) 每种基础类型对应专属包装类：Byte/Short/Integer/Long/Float/Double/Character/Boolean java 运行 String sb = Byte.toString(b); String ss = Short.toString(s); String si = Integer.toString(i); String sl = Long.toString(l); String sf = Float.toString(f); String sd = Double.toString(d); String sc = Character.toString(c); String sbl = Boolean.toString(bl); 方案 4：装箱后调用对象 .toString()（冗余，不推荐） 基础类型自动装箱为包装对象后调用方法，基础类型本身无法调用 .toString() java 运行 int num = 10; Integer obj = num; // 自动装箱 String str = obj.toString();（二）String → 对应基本数据类型（读取输入文本场景） 通用规则：包装类.parseXxx(字符串)，返回基础数据类型 java 运行 String numStr = "123"; String boolStr = "true"; String charStr = "a";

// 1. String → byte byte b = Byte.parseByte(numStr); // 2. String → short short s = Short.parseShort(numStr); // 3. String → int（计算器核心） int i = Integer.parseInt(numStr); // 4. String → long long l = Long.parseLong(numStr); // 5. String → float float f = Float.parseFloat("3.14"); // 6. String → double double d = Double.parseDouble("6.66"); // 7. String → boolean boolean bl = Boolean.parseBoolean(boolStr); // 8. String → char（无 parseChar 方法，特殊处理） char c = charStr.charAt(0); // 取字符串第一个字符，要求字符串长度为1 备选方案：Xxx.valueOf(String)，返回包装对象，自动拆箱 底层本质依旧调用 parseXxx 方法，多一步装箱操作 java 运行 int num = Integer.valueOf("123"); double d = Double.valueOf("2.5");

三、特殊类型转换注意事项

char 字符类型 不存在 Character.parseChar() 方法； 字符串转字符只能使用 str.charAt(0)，要求输入字符串长度必须为 1。 boolean 布尔类型 仅当字符串严格等于 "true"（大小写不敏感）时，转换结果为 true； 其余任意字符串（如 "abc"、"123"）转换结果均为 false。 浮点型 float/double 传入的字符串必须是合法数字 / 小数格式，否则抛出 NumberFormatException。 整数范围限制 byte /short/int /long Byte.parseByte(): 数字范围仅 -128 ~ 127，超出范围抛异常； Short.parseShort(): 范围 -32768 ~ 32767； 超出对应类型取值范围时，会抛出数字格式异常。

四、做题速记最优写法

1. 基础类型转字符串（文本框 setText） 作业简写：变量 + "" 规范工程写法：String.valueOf(变量)
2. 字符串转基础数字（读取输入框 getText） 统一使用 parse 系列方法： 整数：Integer.parseInt(str) 小数：Double.parseDouble(str)
3. 高频避坑提醒 基础类型（int/byte/short 等）不能直接调用 .toString(); char 无 parse 转换方法，使用 charAt(0)； 非数字字符串转数字会直接抛出异常，复杂项目需要加 try-catch 捕获。 你可以直接复制以上全部内容保存为 数据类型转换总结.md 文件使用。

Java 异常「给代码写输出」开卷速查

用法:先用【第一部分】四条铁律判断程序走向,再用【第二部分】查具体异常的 +e / getMessage() 输出,最后用【第三部分】套各种结构(finally/嵌套/多catch/自定义/throws链)的固定输出模式。

第一部分:写输出前先过的铁律

1. "xxx"+e 或 println(e) → 打印 e.toString() → 全限定类名 + ":" + 消息 例: java.lang.ArrayIndexOutOfBoundsException: 10 消息为 null 时只打类名,无冒号。
2. e.getMessage() → 只有消息体,无类名。消息可能是 null (直接打印出 "null")。
3. e.getClass().getName() → 只有类名 java.lang.XxxException;getSimpleName() → 只有 XxxException (无包名)。
4. 程序走向 - 异常被 catch 接住 → 不崩,catch 后代码照常执行。 - 没有匹配的 catch → 异常上抛,当前方法终止,沿调用链找;到 main 还没人接 → 控制台打印 stack trace,程序退出码非0,该异常点之后所有正常代码都不执行。 - try 里抛异常那行之后的语句:跳过,不执行。 - finally 永远执行 (除非 System.exit() 或 JVM 崩溃),哪怕 try/catch 里有 return。

第二部分:常见异常 toString / getMessage 输出对照

△ 版本差异:数组/字符串越界的消息文本在 Java 8 与新版本不同。以你们考试标准答案为准。你给的标答：10 是「只取下标」的写法(老版本风格),下表「标答列」按此填,「新版本」另列。

触发代码	异常全名	toString()	getMessage()
arr[10]=7; (len 3)	java.lang.ArrayIndexOutOfBoundsException	...ArrayIndexOutOfBoundsException: 10	老:10 ;新: Index 10 out of bounds for length 3
5/0 (整数)	java.lang.ArithmeticException	...ArithmeticException: / by zero	/ by zero
Integer.parseInt("abc")	java.lang.NumberFormatException	...NumberFormatException: For input string: "abc"	For input string: "abc"
s=null; s.length()	java.lang.NullPointerException	老:...NullPointerException (无消息,无冒号);新(14+):...NullPointerException: Cannot invoke "String.length()" because "s" is null	老:null ;新:那句完整描述
"abc".charAt(10)	java.lang.StringIndexOutOfBoundsException	...StringIndexOutOfBoundsException: ...	老:String index out of range: 10;新: Index 10 out of bounds for length 3
"abc".substring(5)	java.lang.StringIndexOutOfBoundsException	同上	begin 5, end 3, length 3 (新)/ String index out of range: -2 (老)
(Integer)(Object)"x"	java.lang.ClassCastException	...ClassCastException: class java.lang.String cannot be cast to class java.lang.Integer ...	同(去掉前缀类名部分)
new int[-1]	java.lang.NegativeArraySizeException	...NegativeArraySizeException: -1	-1
Object[] o=new String[1]; o[0]=1;	java.lang.ArrayStoreException	...ArrayStoreException: java.lang.Integer	java.lang.Integer
Class.forName("X.Y") 不存在	java.lang.ClassNotFoundException	...ClassNotFoundException: X.Y	X.Y
遍历中改集合	java.util.ConcurrentModificationException	...ConcurrentModificationException	null
不可变集合 add	java.lang.UnsupportedOperationException	...UnsupportedOperationException	null
Thread.sleep 被中断	java.lang.InterruptedExcep ^{tion}	...InterruptedException: sleep interrupted	sleep interrupted
栈溢出(无限递归)	java.lang.StackOverflowError	...StackOverflowError	null

iterator.next() 无元素	java.util.NoSuchElementException	...NoSuchElementException	null
自定义 throw new XxxException("msg")	你的全限定类名	包名.XxxException: msg	msg
自定义 throw new XxxException() (无参)	你的全限定类名	包名.XxxException (无冒号)	null

记忆要点:消息内容 = 「触发值」。越界→下标;parseInt→那个串;cast→源类型→目标类型;数组存类型错→存入对象的类型;自定义→构造器传的字符串。

第三部分:各种代码结构的输出模式

1. try-catch-finally 执行顺序

```
try{ A; throw; B; } catch(e){ C; } finally{ D; } E;
```

输出顺序:**A → C → D → E**(B 被跳过;D 必执行;异常被接住所以 E 也执行)。若**没被 catch 接住:A → D**,然后异常上抛,E 不执行。

2. finally 里有 return / try 里有 return

- try 里 return 前,finally 仍先执行,再真正返回。
- finally 里有 return → 会覆盖 try/catch 的 return 值和待抛异常**(吞掉异常,考点!)

```
try{ return 1; } finally{ return 2; } // 实际返回 2
try{ throw new RuntimeException(); } finally{ return 3; } // 不抛异常,返回3
```

3. 多 catch:从上往下匹配第一个,父类要放后面

```
try{ 5/0; }
catch(ArithmeticException e){ println("算术"); } // ← 命中这个
catch(Exception e){ println("通用"); }
```

输出:算术。**只进一个 catch**,不会继续往下。△ 若父类 Exception 写在子类前面 → **编译错误**(不是运行输出),写"compile error"。

4. 多异常合并 catch(Java 7+)

```
catch(IOException | SQLException e){ ... } // 合法
```

合并的几个类之间**不能有父子关系**,否则编译错误。

5. 嵌套 try:内层没接到的抛给外层

```
try{
    try{ 5/0; } catch(NumberFormatException e){ println("内"); }
    println("内层后"); // 不执行
} catch(ArithmeticException e){ println("外"); }
```

输出:外 (内层 catch 类型不匹配,异常穿透到外层,"内层后"被跳过)。

6. 异常在方法间传播(throws 链)

```
void a() throws Exception { b(); println("a尾"); } // a尾不执行
void b() throws Exception { throw new Exception("x"); }
// main: try{ a(); } catch(Exception e){ println(e.getMessage()); }
```

输出:**x**。b 抛出→a 不接(只 throws)→a 的"a尾"不执行→传到 main 的 catch。

7. 异常链 getCause()

```
catch(Exception e){
    println(e.getMessage()); // 当前异常消息
    println(e.getCause()); // 底层异常的 toString,没有则 null
}
// 抛出方:throw new RuntimeException("外层", new IOException("内层"));
```

getMessage() → 外层;getCause() → java.io.IOException: 内层。

8. printStackTrace 输出形式(题目若问"打印什么")

```
java.lang.ArithmeticException: / by zero
    at 类名.方法(文件:行号)
    at ...
```

第一行 = toString();后面每行 \tat ... 是栈帧。考试通常只要求写第一行 + 「后调用用栈」即可。

9. 自定义异常完整例

```
class MyException extends Exception {
    public MyException(String m){ super(m); }
}
// throw new MyException("出错了");
// catch(MyException e){
//     println(e);           → 包名.MyException: 出错了
//     println(e.getMessage()); → 出错了
// }
```

super(m) 传进去的就是 message;没传则 getMessage() 为 null。

10. checked vs unchecked(影响"编译能否过")

- Checked**(IOException, SQLException, ClassNotFoundException, InterruptedException, 自定义 extends Exception):必须 try-catch 或 throws,否则**编译错误**。
- Unchecked**(RuntimeException 及其子类:NPE, 各种越界, ArithmeticException, ClassCast, NumberFormat...):不强制处理,不写也能编译,运行时才抛。
- 题目让你判断"能否编译":checked 异常没处理 → compile error;unchecked → 编译通过,运行时输出异常。

第四部分:套你那道原题(对照验证)

```
int arr[]=new int[3];
arr[10]=7;           // 抛 AI00BE,赋值未成功,跳进catch
```

```
// catch:
println("index out of bound!!");    // 字面量
println("Exception="+e);           // "Exception=" + e.toString()
// catch后:
println("end of main()");           // 异常被接住,程序没崩,继续
```

输出:

```
index out of bound!!
Exception=java.lang.ArrayIndexOutOfBoundsException: 10
end of main()
```

核对三处:① 带包名 java.lang. ② 是下标 10 不是长度 3 ③ 最后一行必出。

一页极简口诀

- +e = 类名:消息 | getMessage() = 只消息(可能 null)
- 消息 = 触发值(下标/串/类型)
- catch 接住→后面照跑 | 没接住→方法终止+栈轨迹
- finally 必执行,finally 里 return 会吞异常和覆盖返回值
- 多catch 只进一个;父类在子类前=编译错
- checked 不处理=编译错;unchecked 不处理=运行时抛
- 嵌套/throws:本层不接就往外抛,抛出点后的语句全跳过

Swing GUI 公式化速查(开卷直接拼)

用法:GUI 程序 = 骨架 + 布局 + 组件 + 事件,四层各挑一块往里套。 先抄【第0部分骨架】,再按题目要求从后面各部分挑零件填进去。

第 0 部分:万能骨架(任何问题先抄这个)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class Demo extends JFrame {
    public Demo() {
        setTitle("标题");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 关闭窗口退出
        setLocationRelativeTo(null); // 窗口居中
        setLayout(new BorderLayout()); // ← 这里换布局
        // ← 这里 new 组件、add 进去
        // ← 这里加事件监听
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Demo().setVisible(true));
    }
}
```

四句固定开头的作用(常考填空):

- setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) 不写 → 点 X 窗口消失但程序不退出。
- setLocationRelativeTo(null) → 居中;不写 → 左上角。
- setVisible(true) 必须最后调,且不写窗口不显示。
- setSize 或 pack() 二选一;pack() 按组件首选大小自适应。

第 1 部分:布局(setLayout 换这一行)→ 决定组件怎么排

想要的效果	套这行	add 方式
东南西北中 五区	setLayout(new BorderLayout(水平间距,垂直间距))	add(组件, BorderLayout.NORTH/SOUTH/EAST/WEST/CENTER)
一行从左到右排,挤满换行	setLayout(new FlowLayout())	add(组件) 直接加,按顺序排
网格 m行n列,每格等大	setLayout(new GridLayout(行,列,水平间距,垂直间距))	add(组件) 按从左到右、从上到下填
不设布局,绝对定位	setLayout(null)	组件需 setBounds(x,y,w,h)
卡片堆叠(一次显一张)	setLayout(new CardLayout())	配合 show() 切换

默认布局(不写 setLayout 时):

- JFrame 默认 BorderLayout
- JPanel 默认 FlowLayout

BorderLayout 要点:

- 每个区只能放一个组件;放多个要先塞进 JPanel。
- CENTER 会吸收所有剩余空间;NORTH/SOUTH 占满宽度、高度由组件定;EAST/WEST 反之。
- 只 add 到 CENTER 不写区域 = add(c) 默认就是 CENTER。

第 2 部分:嵌套套路(复杂界面=面板套面板)

核心公式:外层 BorderLayout 分大区,每个大区放一个 JPanel,JPanel 内部再用自己的布局排组件。

```
// 上面一行标签 + 中间大文本 + 下面输入行,就是你那道题的结构:
add(new JLabel("标题"), BorderLayout.NORTH); // 上:单个组件直接放
JTextArea ta = new JTextArea();
add(new JScrollPane(ta), BorderLayout.CENTER); // 中:文本域要套滚动条!
JPanel bottom = new JPanel(new FlowLayout()); // 下:多个组件→先建面板
bottom.add(new JTextField(20));
bottom.add(new JButton("添加"));
add(bottom, BorderLayout.SOUTH);
```

通用嵌套模板:

```
JPanel 子面板 = new JPanel(new 某布局());
子面板.add(组件1);
子面板.add(组件2);
add(子面板, BorderLayout.某区); // 子面板作为一个整体放进主框架的某区
```

第 3 部分:常用组件(new 出来 + 取值/设值)

组件	创建	取内容	设内容
标签	new JLabel("文字")	getText()	setText("x")
单行输入框	new JTextField(列数)	getText()	setText("x")
密码框	new JPasswordField(列数)	getPassword() (返回 char[])	—
多行文本域	new JTextArea(行,列)	getText()	setText / append("x\n") 追加
按钮	new JButton("文字")	—	setText
复选框	new JCheckBox("文字")	isSelected() (boolean)	setSelected(true)
单选按钮	new JRadioButton("文字")	isSelected()	需加进 ButtonGroup 才互斥
下拉框	new JComboBox<>(数组)	getSelectedItem() / getSelectedIndex()	addItem("x")
列表	new JList<>(数组)	getSelectedValue()	—

两个必背陷阱:

- JTextArea 必须包 JScrollPane 才有滚动条:**add(new JScrollPane(textArea), ...),直接 add textArea 不滚。
- JRadioButton 要互斥:**必须建 ButtonGroup bg = new ButtonGroup(); bg.add(r1); bg.add(r2); ,否则能同时选中。ButtonGroup 只管逻辑互斥,**不负责布局**,组件还要单独 add 到面板。

第 4 部分:事件监听(四种写法,任选)

事件 = 给组件 add 一个监听器,组件被操作时执行 actionPerformed。

写法 A:Lambda(最简,Java 8+,推荐开卷用)

```
btn.addActionListener(e -> {
    String s = textField.getText();
    textArea.append(s + "\n");
});
```

写法 B:匿名内部类(老教材最常见)

```
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        textArea.append(textField.getText() + "\n");
    }
});
```

写法 C:让主类 implements ActionListener

```
public class Demo extends JFrame implements ActionListener {
    JButton btn;
    public Demo(){
        btn = new JButton("点");
        btn.addActionListener(this); // ← 注册 this
        add(btn);
    }
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btn) { ... } // 多组件时用 getSource 区分
    }
}
```

写法 D:单独的监听器类

```
class MyListener implements ActionListener {
    public void actionPerformed(ActionEvent e){ ... }
}
// btn.addActionListener(new MyListener());
```

事件挂哪个组件 = 哪个动作触发:

想响应	挂在	监听器	回调方法
按钮点击	JButton	ActionListener	actionPerformed
文本框回车	JTextField	ActionListener	actionPerformed
复选/单选/下拉 改变	对应组件	ActionListener 或 ItemListener	actionPerformed / itemStateChanged
鼠标点击/进入/离开	任意组件	MouseListener (或 MouseAdapter)	mouseClicked 等
键盘按键	任意组件	KeyListener (或 KeyAdapter)	keyPressed 等
窗口关闭	JFrame	WindowListener (或 WindowAdapter)	windowClosing

复用同一段逻辑挂给多个组件(你那题的做法):

```
Runnable task = () -> { /* 共同逻辑 */ };
textField.addActionListener(e -> task.run()); // 回车
btn.addActionListener(e -> task.run()); // 点击
```

Adapter 的意义:XxxListener 接口有多个方法必须全实现;XxxAdapter 是空实现的类,继承它只重写你要的那个方法,省代码。例:只想要鼠标点击 → addMouseListener(new MouseAdapter(){ public void mouseClicked(MouseEvent e){...} })。

第 5 部分:三种典型界面「整套」配方

配方①:输入区 + 显示区(你那道题)

```
布局: BorderLayout
NORTH : JLabel 提示
CENTER : JScrollPane(JTextArea) 显示
SOUTH : JPanel(FlowLayout){ JTextField + JButton }
事件 : 按钮 + 文本框回车 共用一段 append 逻辑
```

配方②:登录框

```
布局: GridLayout(3,2) 或 BorderLayout+面板
行1: JLabel("用户名") JTextField
行2: JLabel("密码") JPasswordField
行3: JButton("登录") JButton("取消")
事件: 登录按钮读取两框内容做判断
```

配方③:计算器/多按钮网格

```
布局: BorderLayout
NORTH : JTextField(显示结果, setEditable(false))
CENTER: JPanel(GridLayout(4,4)){ 16个JButton }
事件 : implements ActionListener,用 e.getSource()
      或 e.getActionCommand() 取按钮文字区分
```

一页极简口诀

- **骨架**:title→size→close→center→layout→加组件→加事件→main 里 invokeLater + setVisible(true)
- **布局**:东南西北中=Border | 一行流=Flow | 网格=Grid | JFrame默认Border,JPanel默认Flow
- **嵌套**:一个区放多个组件 → 先塞进 JPanel 再 add
- **取值**:文本框/域 getText();勾选 isSelected();下拉 getSelectedItem()
- **两大坑**:JTextArea 要 new JScrollPane(ta) 才滚;JRadioButton 要 ButtonGroup 才互斥
- **事件**:组件.addActionListener(e -> {...});按钮点击和文本框回车都用 ActionListener;多组件用 e.getSource() 区分

Java 代码库全量速查（111 个文件：代码 + 运行结果）

开卷用法：先在下面索引表里按 文件名 / 主题 找到编号 №，再翻到对应卡片看完整代码和运行结果。每张卡片含：源文件名、类型标签、完整代码、运行结果说明。

文件索引

№	文件	所在	类型
1	HelloWorld.java	chp1_intro	Console
2	Rectangle1.java	chp2_oop	Console
3	Rectangle2.java	chp2_oop	Console
4	Rectangle.java	chp2_oop	Other
5	TestRect.java	chp2_oop	Console
6	CharToInt.java	exp3_1	Console
7	Assign.java	exp3_2	Console
8	SomeConstTest.java	exp3_3	Console
9	TestInit.java	exp3_4	Console
10	TestUnary.java	exp3_5	Console
11	WhileDemo.java	exp3_6	Console
12	DoWhileDemo.java	exp3_7	Console
13	ForDemo.java	exp3_8	Console
14	IfElseDemo.java	exp3_9	Console
15	SwitchDemo2.java	exp3_10	Other
16	SwitchDemo.java	exp3_10	Console
17	BreakDemo.java	exp3_11	Console
18	BreakWithLabelDemo.java	exp3_12	Console
19	ContinueDemo.java	exp3_13	Console
20	ContinueWithLabelDemo.java	exp3_14	Console
21	ArrayDemo.java	exp3_15	Console
22	ArrayOfStringsDemo.java	exp3_16	Console
23	ArrayOfArraysDemo1.java	exp3_17	Console
24	ArrayOfArraysDemo2.java	exp3_18	Console
25	NewArrayOfStringsDemo.java	exp3_19	Console
26	AboutString.java	exp3_20	Console
27	ArrayCopyDemo.java	exp3_20	Console
28	ClassAndObject.java	exp4_1	Other
29	UseMyDate.java	exp4_2	Console
30	VarTest.java	exp4_3	Console
31	PassTest.java	exp4_4	Console
32	Calculation.java	exp4_5	Console
33	TestOverloading.java	exp4_6	Console
34	MyDate.java	exp4_7	Console
35	UseDog.java	exp4_8	Console
36	EmployeeTest.java	exp4_9	Console
37	Test.java	exp4_10	Console
38	Outer.java	exp4_11	Console
39	Outer.java	exp4_12	Console
40	Outer.java	exp4_13	Console
41	TestInner.java	exp4_14	Console
42	TestSuper.java	exp4_17	Console
43	Sandwich.java	exp4_18	Console
44	SubClass.java	exp4_18	Console
45	SubInitSample.java	exp4_18	Console
46	TestOverriding.java	exp4_19	Console
47	Shapes.java	exp4_20	Console
48	Shapes_2.java	exp4_21	Console
49	TestStaticVar.java	exp5_1	Console
50	UseGeneral.java	exp5_2	Console
51	TestStaticInit.java	exp5_3	Console
52	Customer.java	exp5_4	Console
53	NewShapes.java	exp5_5	Console
54	TestPackage.java	exp5_6	Console
55	ListTest.java	exp5_7	Console
56	ListTestWithGenerics.java	exp5_8	Console
57	MyBoxTest.java	exp5_9	Console
58	WildcardsTest.java	exp5_10	Console
59	BoxTest.java	exp5_11	Console
60	FindDups.java	exp5_12	Console
61	Deal.java	exp5_13	Console
62	Counter.java	exp5_14	Console
63	Freq.java	exp5_15	Console
64	ListIteratorDemo.java	exp5_16	Console
65	EnumValuesTest.java	exp5_18	Other
66	CoinTest2.java	exp5_19	Console
67	CoinTest.java	exp5_19	Console
68	AutoBoxingTest.java	exp5_20	Console

69	HelloWorld.java	exp6_1	Console
70	TestListOfNumbers1.java	exp6_2	FileIO
71	TestListOfNumbers.java	exp6_3	FileIO
72	HelloWorld2.java	exp6_4	Console
73	TestOfDeclared.java	exp6_5	FileIO
74	TestMyException.java	exp6_6	Other
75	AssertionDemo.java	exp6_7	Console
76	CopyBytes.java	exp7_1	FileIO
77	Copy.java	exp7_2	FileIO
78	ReverseThread.java	exp7_3	FileIO
79	RhymingWords.java	exp7_3	FileIO
80	SortThread.java	exp7_3	FileIO
81	DataIOTest.java	exp7_4	FileIO
82	StandardIO.java	exp7_5	FileIO
83	RenameFile.java	exp7_6	Console
84	RandomAccessTest.java	exp7_7	FileIO
85	SerializeDate.java	exp7_8	FileIO
86	UnSerializeDate.java	exp7_9	FileIO
87	ObjectSerial.java	exp7_10	FileIO
88	ObjectTest.java	exp7_11	FileIO
89	CopyBytesWithChannel.java	exp7_12	FileIO
90	HelloWorldSwing.java	exp8_1	GUI
91	JFrameDemo.java	exp8_2	GUI
92	FrameWithPanel.java	exp8_3	GUI
93	FlowLayoutWindow.java	exp8_4	GUI
94	BorderLayoutWindow.java	exp8_5	GUI
95	GridLayoutWindow.java	exp8_6	GUI
96	CardLayoutWindow.java	exp8_7	GUI
97	GridBagLayoutWindow.java	exp8_8	GUI
98	BoxWindow.java	exp8_9	GUI
99	TestButton.java	exp8_10	GUI
100	ComplexListener.java	exp8_11	GUI
101	MultiListener.java	exp8_12	GUI
102	MyFrameCanExit2.java	exp8_13	GUI
103	MyFrameCanExit3.java	exp8_14	GUI
104	TabbedPaneDemo.java	exp8_15	GUI
105	TableDemo.java	exp8_16	GUI
106	TreeDemo.java	exp8_17	GUI
107	Demo8_18.java	exp8_18	GUI
108	JDBCExp.java	chp12_JDBC	JDBC
109	JdbcTest.java	exp12_1	JDBC
110	PreparedTest.java	exp12_2	JDBC
111	TransactionTest.java	exp12_3	JDBC

Nº1 HelloWorld.java

Console

chp1_intro

```
package chp1_intro;

public class HelloWorld{
    public static void main(String[] args){
        System.out.println("Hello World!");
    }
}
```

► 运行结果：

控制台输出(含字面量行):
Hello World!

Nº2 Rectangle1.java

Console

chp2_oop

```
//面向对象
package chp2_oop;

class Rectangle1 {
    static int perimeter(int length, int width) {
        return 2 * (length + width);
    }

    static int area(int length, int width) {
        return length * width;
    }

    public static void main(String[] args) {
        System.out.println("perimeter = " + Rectangle1.perimeter(5, 4));
        System.out.println("area = " + Rectangle1.area(5, 4));
    }
}
```

► 运行结果：

控制台按 main 逻辑输出（含变量计算结果）。

Nº3 Rectangle2.java

Console

chp2_oop

```
//面向对象
package chp2_oop;

class Rectangle2 {
    private int l, w;

    int perimeter() {
        return 2 * (l + w);
    }

    int area() {
        return l * w;
    }

    public static void main(String[] args) {
        Rectangle2 rect1 = new Rectangle2();
        Rectangle2 rect2 = new Rectangle2();
        rect1.l = 10;
        rect1.w = 5;
        System.out.println("perimeter of rect1 = " + rect1.perimeter());
        System.out.println("area of rect1 = " + rect1.area());
        rect2.l = 6;
        rect2.w = 4;
        System.out.println("perimeter of rect2 = " + rect2.perimeter());
        System.out.println("area of rect2 = " + rect2.area());
    }
}
```

► 运行结果：

控制台按 main 逻辑输出（含变量计算结果）。

Nº4 Rectangle.java

Other

chp2_oop

```
package chp2_oop;

class Rectangle {
    private int l, w;

    int perimeter() {
        return 2 * (l + w);
    }

    int area() {
        return l * w;
    }

    Rectangle(int l, int w) {
        this.l = l;
        this.w = w;
    }
}
```

► 运行结果：

演示性代码（类/接口定义或片段），无独立 main 输出。

```
package chp2_oop;

public class TestRect {
    public static void main(String[] args) {
        Rectangle rect1 = new Rectangle(10, 5);
        Rectangle rect2 = new Rectangle(6, 4);
        System.out.println("perimeter of rect1 = " + rect1.perimeter());
        System.out.println("area of rect1 = " + rect1.area());
        System.out.println("perimeter of rect2 = " + rect2.perimeter());
        System.out.println("area of rect2 = " + rect2.area());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

实验3 类·对象·数组·语法 (exp3)

№6 CharToInt.java Console

exp3_1

```
package chp3_base.exp3_1;
public class CharToInt {
    public static void main(String[] args) {
        int intResult, intVar = 10;
        char charVar = '语';
        intResult = intVar + charVar;
        System.out.println("The char is: " + charVar);
        System.out.println("The char's Unicode is: \\u"
            + Integer.toHexString(charVar));
        System.out.println("The int value corresponding to the char is: "
            + new Integer(charVar).toString());
        System.out.println("Int " + intVar + " adds the char, the result is: "
            + intResult);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№7 Assign.java Console

exp3_2

```
package chp3_base.exp3_2;
public class Assign{
    public static void main (String[] args){
        int x,y;           //声明int型变量
        float z=3.414f;     //声明并赋值float型变量
        double w=3.1415;    //声明并赋值double型变量
        boolean truth=true; //声明并赋值boolean型变量
        char c;             //声明char型变量
        String str;         //声明String类变量
        String str1="bye";  //声明并赋值String类变量
        c='A';              //给char型变量赋值
        str="Hi out there"; //给String类变量赋值
        x=6;                //给int型变量赋值
        y=1000;
        System.out.println("x = "+x);
        System.out.println("y = "+y);
        System.out.println("z = "+z);
        System.out.println("w = "+w);
        System.out.println("truth = "+truth);
        System.out.println("c = "+c);
        System.out.println("str = "+str);
        System.out.println("str1 = "+str1);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№8 SomeConstTest.java Console

exp3_3

```
package chp3_base.exp3_3;
public class SomeConstTest {
    public static void main(String[] args) {
        // 输出byte型的最大值与最小值
        System.out.println("Byte.MAX_VALUE = " + Byte.MAX_VALUE);
        System.out.println("Byte.MIN_VALUE = " + Byte.MIN_VALUE + '\n');
        System.out.println();
        // 输出short型的最大值与最小值
        System.out.println("Short.MAX_VALUE = " + Short.MAX_VALUE);
        System.out.println("Short.MIN_VALUE = " + Short.MIN_VALUE + '\n');
        // 输出int型的最大值与最小值
        System.out.println("Integer.MAX_VALUE = " + Integer.MAX_VALUE);
        System.out.println("Integer.MIN_VALUE = " + Integer.MIN_VALUE + '\n');
        // 输出long型的最大值与最小值
        System.out.println("Long.MAX_VALUE = " + Long.MAX_VALUE);
        System.out.println("Long.MIN_VALUE = " + Long.MIN_VALUE + '\n');
        // 输出float型的最大值与最小值
        System.out.println("Float.MAX_VALUE = " + Float.MAX_VALUE);
        System.out.println("Float.MIN_VALUE = " + Float.MIN_VALUE + '\n');
        // 输出double型的最大值与最小值
        System.out.println("Double.MAX_VALUE = " + Double.MAX_VALUE);
        System.out.println("Double.MIN_VALUE = " + Double.MIN_VALUE + '\n');
        // 输出float型的正无穷大与负无穷大
        System.out.println("Float.POSITIVE_INFINITY = " + Float.POSITIVE_INFINITY);
        System.out.println("Float.NEGATIVE_INFINITY = " + Float.NEGATIVE_INFINITY + '\n');
        // 输出double型的正无穷大与负无穷大
        System.out.println("Double.POSITIVE_INFINITY = " + Double.POSITIVE_INFINITY);
        System.out.println("Double.NEGATIVE_INFINITY = " + Double.NEGATIVE_INFINITY + '\n');
        // 输出float型0/0
        System.out.println("Float.NaN = " + Float.NaN + '\n');
        // 输出double型0/0
        System.out.println("Double.NaN = " + Double.NaN + '\n');
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№9 TestInit.java Console

exp3_4

```
package chp3_base.exp3_4;
public class TestInit {
    int x=0;
    public static void main(String[] args) {
        TestInit init = new TestInit();
        int x = (int) (Math.random() * 100);
        int z;
        int y=0;
        if (x > 50) {
            y = 9;
        }
        z = x + y + init.x;
        System.out.println("x=" + x + " y=" + y + " z=" + z);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№10 TestUnary.java Console

exp3_5

```
package chp3_base.exp3_5;
public class TestUnary {
    public static void main(String[] args) {
        int a = 9;
        int b = -a;
        byte bb = 9;
        int ib = +bb;
        int x = 4, y = 8;
        int z;
        int i = 0;
        int j = i++;
        int k = ++j;
        z = (x++) * (--y);
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("bb = " + bb);
        System.out.println("ib = " + ib);
        System.out.println("i = " + i);
        System.out.println("j = " + j);
        System.out.println("k = " + k);
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("z = " + z);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№11 WhileDemo.java Console

exp3_6

```
package chp3_base.exp3_6;
public class WhileDemo {
    public static void main(String[] args) {
        String copyFromMe = "Copy this string until you encounter the letter 'g'";
        StringBuffer copyToMe = new StringBuffer();
        int i = 0;
        char c = copyFromMe.charAt(i);
        while (c != 'g') {
            copyToMe.append(c);
            c = copyFromMe.charAt(++i);
        }
        System.out.println(copyToMe);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№12 DoWhileDemo.java Console

exp3_7

```
package chp3_base.exp3_7;
public class DoWhileDemo {
    public static void main(String[] args) {
        String copyFromMe = "Copy this string until you encounter the letter 'g'";
        StringBuffer copyToMe = new StringBuffer();
        int i = 0;
        char c = copyFromMe.charAt(i);
        do {
            copyToMe.append(c);
            c = copyFromMe.charAt(++i);
        } while (c != 'g');
        System.out.println(copyToMe);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№13 ForDemo.java

Console

exp3_8

```
package chp3_base.exp3_8;
public class ForDemo {
    public static void main(String[] args) {
        int[] arrayOfInts = { 32, 87, 3, 589, 12, 1076, 2000, 8, 622, 127 };
        for (int i = 0; i < arrayOfInts.length; i++) {
            System.out.print(arrayOfInts[i] + " ");
        }
        System.out.println();
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№14 IfElseDemo.java

Console

exp3_9

```
package chp3_base.exp3_9;
public class IfElseDemo {
    public static void main(String[] args) {
        int testScore = 76;
        char grade;
        if (testScore >= 90) {
            grade = 'A';
        } else if (testScore >= 80) {
            grade = 'B';
        } else if (testScore >= 70) {
            grade = 'C';
        } else if (testScore >= 60) {
            grade = 'D';
        } else {
            grade = 'E';
        }
        System.out.println("Grade = " + grade);
        System.out.println("The character "+grade+" is "+
            (Character.isUpperCase(grade)?"upper":"lower")+"case.");
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№15 SwitchDemo2.java

Other

exp3_10

```
package chp3_base.exp3_10;
public class SwitchDemo2 {
    public static void main(String[] args) {
        int month = 2;
        int year = 2000;
        int numDays = 0;
        switch (month) {
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
                numDays = 31;
                break;
            case 4:
            case 6:
            case 9:
            case 11:
                numDays = 30;
                break;
            case 2:
                if (((year % 4 == 0) && !(year % 100 == 0)) || (year % 400 == 0))
                    numDays = 29;
                else
                    numDays = 28;
                break;
        }
        System.out.println("The date is 2000.2. The number of Days = "
            + numDays);
    }
}
```

► 运行结果：演示性代码（类/接口定义或片段），无独立 main 输出。

```
package chp3_base.exp3_10;
public class SwitchDemo {
    public static void main(String[] args) {
        int month = 8;
        switch (month) {
            case 1:
                System.out.println("January");
                break;
            case 2:
                System.out.println("February");
                break;
            case 3:
                System.out.println("March");
                break;
            case 4:
                System.out.println("April");
                break;
            case 5:
                System.out.println("May");
                break;
            case 6:
                System.out.println("June");
                break;
            case 7:
                System.out.println("July");
                break;
            case 8:
                System.out.println("August");
                break;
            case 9:
                System.out.println("September");
                break;
            case 10:
                System.out.println("October");
                break;
            case 11:
                System.out.println("November");
                break;
            case 12:
                System.out.println("December");
                break;
            default:
                System.out.println("Hey, that's not a valid month!");
                break;
        }
    }
}
```

► 运行结果：控制台输出(含字面量行)：

```
January
February
March
April
May
June
July
August
September
October
November
December
...
```

```
package chp3_base.exp3_11;
public class BreakDemo {
    public static void main(String[] args) {
        int[] arrayOfInts = { 32, 87, 3, 589, 12, 1076, 2000, 8, 622, 127 };
        int searchFor = 12;
        int i = 0;
        boolean foundIt = false;
        for (; i < arrayOfInts.length; i++) {
            if (arrayOfInts[i] == searchFor) {
                foundIt = true;
                break;
            }
        }
        if (foundIt) {
            System.out.println("Found " + searchFor + " at index " + i);
        } else {
            System.out.println(searchFor + " not in the array");
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№18 BreakWithLabelDemo.java Console

exp3_12

```
package chp3_base.exp3_12;
public class BreakWithLabelDemo {
    public static void main(String[] args) {
        int[][] arrayOfInts = { { 32, 87, 3, 589 }, { 12, 1076, 2000, 8 },
                                { 622, 127, 77, 955 } };
        int searchFor = 12;
        int i = 0;
        int j = 0;
        boolean foundIt = false;
        search: for (; i < arrayOfInts.length; i++) {
            for (j = 0; j < arrayOfInts[i].length; j++) {
                if (arrayOfInts[i][j] == searchFor) {
                    foundIt = true;
                    break search;
                }
            }
        }
        if (foundIt) {
            System.out.println("Found " + searchFor + " at " + i + ", " + j);
        } else {
            System.out.println(searchFor + "not in the array");
        }
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№19 ContinueDemo.java Console

exp3_13

```
package chp3_base.exp3_13;
public class ContinueDemo {
    public static void main(String[] args) {
        StringBuffer searchMe = new StringBuffer(
            "peter piper picked a peck of pickled peppers");
        int max = searchMe.length();
        int numPs = 0;
        for (int i = 0; i < max; i++) {
            if (searchMe.charAt(i) != 'p')
                continue;
            numPs++;
            searchMe.setCharAt(i, 'P');
        }
        System.out.println("Found " + numPs + " p's in the string.");
        System.out.println(searchMe);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№20 ContinueWithLabelDemo.java Console

exp3_14

```
package chp3_base.exp3_14;
public class ContinueWithLabelDemo {
    public static void main(String[] args) {
        String searchMe = "Look for a substring in me";
        String subString = "sub";
        boolean foundIt = false;
        int max = searchMe.length() - subString.length();
        test: for (int i = 0; i < max; i++) {
            int n = subString.length();
            int j = i;
            int k = 0;
            while (n-- != 0) {
                if (searchMe.charAt(j++) != subString.charAt(k++)) {
                    continue test;
                }
            }
            foundIt = true;
            break test;
        }
        System.out.println(foundIt ? "Found it" : "Didn't find it");
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№21 ArrayDemo.java Console

exp3_15

```
package chp3_base.exp3_15;
public class ArrayDemo {
    public static void main(String[] args) {
        int[] anArray;           //声明一个整形数组
        anArray = new int[10];    //创建数组
        //给数组每个元素赋值并打印输出
        for (int i = 0; i < anArray.length; i++) {
            anArray[i] = i;
            System.out.print(anArray[i] + " ");
        }
        System.out.println();
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№22 ArrayOfStringsDemo.java

Console

exp3_16

```
package chp3_base.exp3_16;
public class ArrayOfStringsDemo {
    public static void main(String[] args) {
        String[] anArray = { "String One", "String Two", "String Three" };
        for (int i = 0; i < anArray.length; i++) {
            System.out.println(anArray[i].toLowerCase());
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№23 ArrayOfArraysDemo1.java

Console

exp3_17

```
package chp3_base.exp3_17;
public class ArrayOfArraysDemo1 {
    public static void main(String[] args) {
        int[][] aMatrix = new int[4][]; // aMatrix指向4个引用组成的元素
        // 创建每个int数组，并进行赋值
        for (int i = 0; i < aMatrix.length; i++) {
            aMatrix[i] = new int[5];
            for (int j = 0; j < aMatrix[i].length; j++) {
                aMatrix[i][j] = i + j;
            }
        }
        // 将数组打印输出
        for (int i = 0; i < aMatrix.length; i++) {
            for (int j = 0; j < aMatrix[i].length; j++) {
                System.out.print(aMatrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№24 ArrayOfArraysDemo2.java

Console

exp3_18

```
package chp3_base.exp3_18;
public class ArrayOfArraysDemo2 {
    public static void main(String[] args) {
        String[][] cartoons = {
            { "Flintstones", "Fred", "Wilma", "Pebbles", "Dino" },
            { "Rubbles", "Barney", "Betty", "Bam Bam" },
            { "Jetsons", "George", "Jane", "Elroy", "Judy", "Rosie", "Astro" },
            { "Scooby Doo Gang", "Scooby Doo", "Shaggy", "Velma", "Fred", "Daphne" } };
        for (int i = 0; i < cartoons.length; i++) {
            System.out.print("cartoons[" + i + "]" + ", " + cartoons[i].length
                + " elements: ");
            for (int j = 0; j < cartoons[i].length; j++) {
                System.out.print(cartoons[i][j]
                    + (j < cartoons[i].length - 1 ? ", " : ""));
            }
            System.out.println();
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№25 NewArrayOfStringsDemo.java

Console

exp3_19

```
package chp3_base.exp3_19;
public class NewArrayOfStringsDemo {
    public static void main(String[] args) {
        String[] anArray = { "String One", "String Two", "String Three" };
        for (String s : anArray) {
            System.out.println(s.toLowerCase());
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp3_base.exp3_20;

public class AboutString {
    public static void main(String[] args) {
        String a = "HUST", b = a + ".AIA.Java";
        String c = "HUST" + ".AIA.Java", d = "HUST.AIA" + ".Java";
        String e = new String("HUST.AIA.Java");
        if (b == c) {
            System.out.println("苟利国家生死以，岂因祸福避趋之。");
        }
        if (c == d) {
            System.out.println("为有牺牲多壮志，敢教日月换新天。");
        }
        if (d == e) {
            System.out.println("人生自古谁无死，留取丹心照汗青。");
        }
        System.out.println("人民、只有人民，才是创造世界历史的动力。");
    }
}
```

► 运行结果：控制台输出(含字面量行)：

苟利国家生死以，岂因祸福避趋之。

为有牺牲多壮志，敢教日月换新天。

人生自古谁无死，留取丹心照汗青。

人民、只有人民，才是创造世界历史的动力。

```
package chp3_base.exp3_20;

public class ArrayCopyDemo {
    public static void main(String[] args) {
        char[] copyFrom={'d','e','c','a','f','f','e','i','n','a','t','e','d'};
        char[] copyTo=new char[7];
        System.arraycopy(copyFrom, 2, copyTo, 0, 7);
        System.out.println(new String(copyTo));
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

实验4 继承·多态·抽象·接口 (exp4)

№28 ClassAndObject.java Other

exp4_1

```
package chp4_oo.exp4_1;
class EmpInfo{
    String name;
    String designation;
    String department;
    public EmpInfo(String eName, String eDesign, String eDept){
        name = eName;
        designation = eDesign;
        department = eDept;
    }
    void print( ){ System.out.println(name+" is a "+
        designation+" at "+department+".");}
}
public class ClassAndObject{
    public static void main(String args[]){
        EmpInfo e1 = new EmpInfo("Robert Java","Manager","Coffee shop" );
        EmpInfo e2 = new EmpInfo("Tom Java","Worker","Coffee shop" );
        e1.print();
        e2.print();
    }
}
```

► 运行结果：演示性代码（类/接口定义或片段），无独立 main 输出。

№29 UseMyDate.java Console

exp4_2

```
package chp4_oo.exp4_2;
class MyDate {
    private int day;
    private int month;
    private int year;
    public String getDate(){
        return day+"/"+month+"/"+year;
    }
    public int setDate(int a,int b,int c){
        if ((a>0 && a<=31)&&(b>0&&b<=12)){
            day=a;
            month=b;
            year=c;
            return 0;
        }else {
            return -1;
        }
    }
}
public class UseMyDate{
    public static void main(String[] args){
        MyDate d = new MyDate();
        if(d.setDate(22,5,2009)==0){
            System.out.println(d.getDate());
        }
    }
}
/*
//illegal operations.
d.day=22;
d.month = 5 ;
d.year = 2009;
System.out.println(d.day+"/"+d.month+"/"+d.year); */
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№30 VarTest.java Console

exp4_3

```
package chp4_oo.exp4_3;
public class VarTest{
    private int x=1;
    private int y=1;
    private int z=1;
    void changeVar(int a,int b,int c){
        x=a;
        int y = b; //y使同名类成员变量隐藏
        int z = 9; //z使同名类成员变量隐藏
        System.out.println("In changVar : "+"x = "+x+" y = "+y+" z = "+z);
        this.z=c; //给类成员变量z赋值
    }
    String getXYZ(){
        return "x = "+x+" y = "+y+" z = "+z;
    }
    public static void main( String args[] ){
        VarTest v=new VarTest();
        System.out.println("Before changVar : "+v.getXYZ());

        v.changeVar(10,10,10);
        System.out.println("After changeVar: "+v.getXYZ());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp4_oo.exp4_4;

public class PassTest {
    float ptValue;

    // 参数类型是基本类型
    public void changeInt(int value) {
        value = 55;
    }

    // 参数类型是引用型, 并且方法中改了变参数的值
    public void changeStr(String value) {
        value = new String("different");
    }

    // 参数类型是引用型, 并且方法中改了变参数所指向对象的成员变量值
    public void changeObjValue(PassTest ref) {
        ref.ptValue = 99.0f;
    }

    public static void main(String args[]) {
        String str;
        int val;

        PassTest pt = new PassTest(); // 创建PassTest的对象

        // 测试基本类型参数的传递, 观察调用程序中的变量值是否改变
        val = 11;
        pt.changeInt(val);
        System.out.println("Int value is: " + val);

        // 测试引用类型参数的传递, 观察调用程序中的变量值是否改变
        str = new String("Hello");
        pt.changeStr(str);
        System.out.println("Str value is: " + str);

        // 测试引用类型参数的传递, 观察调用程序中的变量值是否改变
        pt.ptValue = 101.0f;
        pt.changeObjValue(pt);
        System.out.println("Pt value is: " + pt.ptValue);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

```
package chp4_oo.exp4_5;

public class Calculation {
    public float avg(int... nums) {
        int sum = 0;
        for ( int x : nums ) {
            sum += x;
        }
        return ((float) sum) / nums.length;
    }

    public static void main(String args[]){
        Calculation cal = new Calculation();
        float average1 = cal.avg(10, 20, 30);
        float average2 = cal.avg(5, 6, 7, 8, 9, 10);
        System.out.println("The average of 10,20,30 is"+" "+average1);
        System.out.println("The average of 5, 6, 7, 8, 9, 10 is"+" "+average2);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№33 TestOverLoading.java Console

exp4_6

```
package chp4_oo.exp4_6;
public class TestOverLoading{
    public static void main(String[] args){
        boolean b=true;
        char c = 'A';
        int i = 100 ;
        long l = 1000;
        float f = 99.0f ;
        double d = 999.0;
        char[] cc = {'A','B','C','D'};
        String s = "abcdefg";
        java.util.Date o = new java.util.Date();
        System.out.println( b );
        System.out.println( c );
        System.out.println( i );
        System.out.println( l );
        System.out.println( f );
        System.out.println( d );
        System.out.println();
        System.out.println( cc );
        System.out.println( s );
        System.out.println( o );
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№34 MyDate.java Console

exp4_7

```
package chp4_oo.exp4_7;
public class MyDate{
    private int day, month, year;
    public MyDate(int day,int month,int year){
        this.day = day;
        this.month = month;
        this.year = year;
    }
    public String tomorrow(){
        day = day+1;
        return day+"/"+month+"/"+year;
    }
    public static void main( String[] args){
        MyDate d = new MyDate(12,4,2009);
        System.out.println(d.tomorrow());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№35 UseDog.java Console

exp4_8

```
package chp4_oo.exp4_8;
class Dog {
    String name;
    Dog(String name) {
        this.name = name;
    }
    String getName() {
        return name;
    }
    void setName(String name) {
        this.name = name;
    }
}
public class UseDog {
    public static void main(String[] args) {
        Dog d = new Dog("旺财");
        d.setName("小强");
        System.out.println("The dog's name is "+d.getName());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№36 EmployeeTest.java Console

exp4_9

```
package chp4_oo.exp4_9;
class Employee{
    private String name;
    private int salary;
    public Employee(String n, int s){
        name = n ;
        salary = s;
    }
    public Employee(String n){
        this(n,0);
    }
    public Employee( ){
        this("Unknown");
    }
    public String getName(){
        return name;
    }
    public int getSalary(){
        return salary;
    }
}
public class EmployeeTest{
    public static void main(String [] args){
        Employee e = new Employee();
        System.out.println("Name: "+e.getName()+"    Salary: "+e.getSalary());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№37 Test.java Console

exp4_10

```
package chp4_oo.exp4_10;
class Alpha {
    private int iamprivate;
    public Alpha( int i){
        iamprivate=i;
    }
    boolean isEqualTo(Alpha anotherAlpha) {
        if (this.iamprivate == anotherAlpha.iamprivate)
            return true;
        else
            return false;
    }
}
public class Test{
    public static void main(String args[]){
        Alpha aa=new Alpha(10);
        Alpha bb=new Alpha(12);
        if(aa.isEqualTo(bb)){
            System.out.println("equal ");
        }
        else{
            System.out.println("not equal ");
        }
    }
}
```

► 运行结果：控制台输出(含字面量行):

equal
not equal

№38 Outer.java Console

exp4_11

```
package chp4_oo.exp4_11;
public class Outer{
    private int size;
    /** 定义内部类Inner */
    public class Inner{
        //将外包类的成员变量size递增
        public void doStuff(){
            size++;
        }
    }
    Inner i=new Inner(); //成员变量i指向Inner类的对象
    public void increaseSize(){
        i.doStuff(); //调用内部类Inner的方法
    }
    public static void main(String[] a){
        Outer o=new Outer();
        for (int i = 0; i<4; i++){
            o.increaseSize();
            System.out.println("The value of size : "+o.size);
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp4_oo.exp4_12;
public class Outer{
    private int size;
    /** 定义内部类Inner */
    public class Inner{
        private int size;
        public void doStuff(int size){
            size++; //存取局部变量
            this.size++; //存取内部类的成员变量
            Outer.this.size++; //存取其外包类Outer的成员变量
            System.out.println("size in Inner.doStuff(): "+size);
            System.out.println("size of the Inner class: "+this.size);
            System.out.println("size of the Outer class: "+Outer.this.size);
        }
    }
    Inner i=new Inner(); //成员变量i指向Inner类的对象
    public void increaseSize(int s){
        i.doStuff(s); //调用内部类Inner的方法
    }
    public static void main(String[] a){
        Outer o=new Outer();
        o.increaseSize(10);
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

```
package chp4_oo.exp4_13;
class Outer{
    private int size=5;
    /** 方法makeInner(), 返回一内部类对象 */
    public Object makeInner(final int finalLocalVar){
        int localVar=6;
        class Inner{
            public String toString(){
                return ("#<Inner size="+size+
                    // " localVar=" + localVar + //此处对局部变量localVar的访问非法
                    " finalLocalVar="+finalLocalVar+">");
            }
        }
        return new Inner(); //方法makeInner()返回一内部类对象
    }
    public static void main(String[] args){
        Outer outer=new Outer ();
        Object obj=outer.makeInner(40);
        System.out.println("The object is "+obj.toString());
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

```
package chp4_oo.exp4_14;

class Outer{
    private int size;

    /** 定义内部类Inner */
    class Inner{
        void doStuff(){
            size++;
            System.out.println("The size value of the Outer class: "+size);
        }
    }
}

public class TestInner{
    public static void main(String[] a){
        Outer out=new Outer();
        Outer.Inner in=out.new Inner(); //声明并创建内部类的对象

        in.doStuff(); //调用内部类的方法
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

№42 TestSuper.java

Console

exp4_17

```
package chp4_oo.exp4_17;
class Empolyee {
    private String name ;
    private int salary;
    public Empolyee(String name,int salary){
        this.name = name;
        this.salary = salary;
    }
    public String getDetails( ){
        return "Name: "+name+"\nSalary: "+salary;
    }
}
class Manager extends Empolyee {
    private String department ;
    public Manager(String name,int salary,String department){
        super(name,salary);
        this.department = department;
    }
    public String getDetails( ){
        return super.getDetails( )+"\nDepartment: "+ department;
    }
}
public class TestSuper{
    public static void main(String[] srgs){
        Manager m = new Manager("TOM",2000,"Finance");
        System.out.println(m.getDetails());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№43 Sandwich.java

Console

exp4_18

```
package chp4_oo.exp4_18;
class Meal {
    static int i=10;
    static {
        System.out.println(++i);
    }
    Meal() {
        System.out.println("Meal()");
    }
}
class Bread {
    Bread() {
        System.out.println("Bread()");
    }
}
class Cheese {
    Cheese() {
        System.out.println("Cheese()");
    }
}
class Lettuce {
    Lettuce() {
        System.out.println("Lettuce()");
    }
}
class Lunch extends Meal {
    Lunch() {
        System.out.println("Lunch()");
    }
}
class PortableLunch extends Lunch {
    PortableLunch() {
        System.out.println("PortableLunch()");
    }
}
public class Sandwich extends PortableLunch {
    private Bread b = new Bread();
    private Cheese c = new Cheese();
    private Lettuce l = new Lettuce();
    public Sandwich() {
        System.out.println("Sandwich()");
    }
    static {
        System.out.println("Static statement");
        new Sandwich();
    }
    public static void main(String[] args) {
        new Sandwich();
    }
}
```

► 运行结果：控制台输出(含字面量行)：

```
Meal()
Bread()
Cheese()
Lettuce()
Lunch()
PortableLunch()
Sandwich()
Static statement
```



```
package chp4_oo.exp4_18;
class Parent {
    // 静态变量
    public static String p_StaticField = "父类--静态变量";
    // 变量
    public String p_Field = "父类--变量";
    protected int i = 9;
    protected int j = 0;

    // 静态初始化块
    static {
        System.out.println(p_StaticField);
        System.out.println("父类--静态初始化块");
    }

    // 初始化块
    {
        System.out.println(p_Field);
        System.out.println("父类--初始化块");
    }

    // 构造器
    public Parent() {
        System.out.println("父类--构造器");
        System.out.println("i=" + i + ", j=" + j);
        j = 20;
    }
}

public class SubClass extends Parent {
    // 静态变量
    public static String s_StaticField = "子类--静态变量";
    // 变量
    public String s_Field = "子类--变量";

    // 静态初始化块
    static {
        System.out.println(s_StaticField);
        System.out.println("子类--静态初始化块");
    }

    // 初始化块
    {
        System.out.println(s_Field);
        System.out.println("子类--初始化块");
    }

    // 构造器
    public SubClass() {
        System.out.println("子类--构造器");
        System.out.println("i=" + i + ", j=" + j);
    }

    // 程序入口
    public static void main(String[] args) {
        System.out.println("子类main方法");
        new SubClass();
    }
}
```

► 运行结果：控制台输出(含字面量行)：

父类--静态初始化块
父类--初始化块
父类--构造器
子类--静态初始化块
子类--初始化块
子类--构造器
子类main方法

```
package chp4_oo.exp4_18;

public class SubInitSample {
    public static void main(String[] args) { new B(); }
    private static class A {
        public static C c = new C("A");
        static { System.out.println("A类static块"); }
        public A() { System.out.println("A的构造方法"); }
        { System.out.println("A类普通块"); }
    }
    private static class B extends A {
        public static C c = new C("B");
        static { System.out.println("B类static块"); }
        { System.out.println("B类普通块"); }
        public B() { System.out.println("B的构造方法"); }
    }
    private static class C {
        String id;
        public C(String id) {
            this.id = id;
            System.out.println(this.id + "调用C的构造方法 "); }
    }
}
```

► 运行结果：控制台输出(含字面量行):

```
A类static块
A的构造方法
A类普通块
B类static块
B类普通块
B的构造方法
```

```
package chp4_oo.exp4_19;

class Employee {
    String name ;
    int salary;
    public Employee(String name,int salary){
        this.name = name;
        this.salary = salary;
    }
    public String getDetails( ){
        return "Name: "+name+"\nSalary: "+salary;
    }
}

class Manager extends Employee {
    private String department ;
    public Manager(String name,int salary,String department){
        super(name,salary);
        this.department = department;
    }
    public String getDetails( ){
        return "Name: "+name+"\nSalary: "+salary+"\nDepartment: "+ department;
    }
}

class Secretary extends Employee{
    public Secretary(String name,int salary){
        super(name,salary);
    }
}

public class TestOverriding{
    public static void main(String[] srgs){
        Manager m = new Manager("Tom",2000,"Finance");
        Secretary s = new Secretary("Mary",1500);
        System.out.println(m.getDetails());
        System.out.println(s.getDetails());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp4_oo.exp4_20;
import java.util.Random;
//定义Shape类
class Shape {
    void draw() {}
    void erase() {}
}
//定义Circle类
class Circle extends Shape {
    void draw() {
        System.out.println("Calling Circle.draw()");
    }
    void erase() {
        System.out.println("Calling Circle.erase()");
    }
}
//定义Square类
class Square extends Shape {
    void draw() {
        System.out.println("Calling Square.draw()");
    }
    void erase() {
        System.out.println("Calling Square.erase()");
    }
}
//定义Triangle类
class Triangle extends Shape {
    void draw() {
        System.out.println("Calling Triangle.draw()");
    }
    void erase() {
        System.out.println("Calling Triangle.erase()");
    }
}
//包含main()的测试类
public class Shapes{
    static void drawOneShape(Shape s){
        s.draw();
    }
    static void drawShapes(Shape[] ss){
        for(int i = 0; i < ss.length; i++){
            ss[i].draw();
        }
    }
}

public static void main(String[] args) {
    Random rand = new Random();
    Shape[] s = new Shape[9];
    for(int i = 0; i < s.length; i++){
        switch(rand.nextInt(3)) {
            case 0: s[i] = new Circle();break;
            case 1: s[i] = new Square();break;
            case 2: s[i] = new Triangle();break;
        }
    }
    drawShapes(s);
}
}
```

► 运行结果：控制台输出(含字面量行)：

```
Calling Circle.draw()
Calling Circle.erase()
Calling Square.draw()
Calling Square.erase()
Calling Triangle.draw()
Calling Triangle.erase()
```

```
package chp4_oo.exp4_21;
import java.util.Random;
//定义Shape类
class Shape {
    void draw() {}
    void erase() {}
}
//定义Circle类
class Circle extends Shape {
    void draw() {
        System.out.println("Calling Circle.draw()");
    }
    void erase() {
        System.out.println("Calling Circle.erase()");
    }
}
//定义Square类
class Square extends Shape {
    void draw() {
        System.out.println("Calling Square.draw()");
    }
    void erase() {
        System.out.println("Calling Square.erase()");
    }
}
//定义Triangle类
class Triangle extends Shape {
    void draw() {
        System.out.println("Calling Triangle.draw()");
    }
    void erase() {
        System.out.println("Calling Triangle.erase()");
    }
}
//定义Pentagon类
class Pentagon extends Shape{
    void draw() {
        System.out.println("Calling Pentagon.draw()");
    }
    void erase() {
        System.out.println("Calling Pentagon.erase()");
    }
}
//包含main()的测试类
public class Shapes_2 {
    static void drawOneShape(Shape s){
        s.draw();
    }
    static void drawShapes(Shape[] ss){
        for(int i = 0; i < ss.length; i++){
            ss[i].draw();
        }
    }
}

public static void main(String[] args) {
    Random rand = new Random();
    Shape[] s = new Shape[9];
    for(int i = 0; i < s.length; i++){
        switch(rand.nextInt(4)) {
            case 0: s[i] = new Circle();break;
            case 1: s[i] = new Square();break;
            case 2: s[i] = new Triangle();break;
            case 3: s[i] = new Pentagon();break;
        }
    }
    drawShapes(s);
}
}
```

► 运行结果：控制台输出(含字面量行):

```
Calling Circle.draw()
Calling Circle.erase()
Calling Square.draw()
Calling Square.erase()
Calling Triangle.draw()
Calling Triangle.erase()
Calling Pentagon.draw()
Calling Pentagon.erase()
```

实验5 集合·泛型·内部类·线程入门 (exp5)

№49 TestStaticVar.java Console

exp5_1

```
package chp5_advanced.exp5_1;
class Count{
    private int serialNumber ;
    public static int counter = 0 ;
    public Count( ){
        counter++;
        serialNumber = counter ;
    }
    public int getSerialNumber(){
        return serialNumber;
    }
}

public class TestStaticVar{
    public static void main(String[] args){
        Count[] cc = new Count[10];
        for(int i=0;i<cc.Length;i++){
            cc[i]=new Count();
            System.out.println("cc["+i+"] .serialNumber = "+cc[i].getSerialNumber());
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№50 UseGeneral.java Console

exp5_2

```
package chp5_advanced.exp5_2;
class GeneralFunction{
    public static int add(int x, int y){
        return x+y ;
    }
}

public class UseGeneral{
    public static void main(String[] args){
        int c = GeneralFunction.add(9,10);
        System.out.println("9 + 10 = "+c);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№51 TestStaticInit.java Console

exp5_3

```
package chp5_advanced.exp5_3;
class StaticInitDemo{
    static int i;
    static {
        i = 5;
        System.out.println("Static code: i="+ i++);
    }
}

public class TestStaticInit {
    public static void main(String args[]){
        System.out.println(" Main code: i="+ StaticInitDemo.i);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№52 Customer.java Console

exp5_4

```
package chp5_advanced.exp5_4;
class Customer{
    private final long customerID;
    private static long counter=200901;
    public Customer(){
        customerID = counter++;
    }
    public long getID(){
        return customerID;
    }
    public static void main(String[] args){
        Customer [] cc = new Customer[5];
        for ( int i=0; i<cc.Length; i++){
            cc[i]=new Customer();
            System.out.println("The customerID is "+cc[i].getID());
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp5_advanced.exp5_5;

import java.util.*;

//将Shape定义为interface
interface Shape {
    void draw();
    void erase();
}

//定义Circle类实现Shape
class Circle implements Shape {
    public void draw() {
        System.out.println("Calling Circle.draw()");
    }
    public void erase() {
        System.out.println("Calling Circle.erase()");
    }
}

//定义Square类实现Shape
class Square implements Shape {
    public void draw() {
        System.out.println("Calling Square.draw()");
    }
    public void erase() {
        System.out.println("Calling Square.erase()");
    }
}

//定义Triangle类实现Shape
class Triangle implements Shape {
    public void draw() {
        System.out.println("Calling Triangle.draw()");
    }
    public void erase() {
        System.out.println("Calling Triangle.erase()");
    }
}

//包含main()的测试类
public class NewShapes{

    static void drawOneShape(Shape s){
        s.draw();
    }

    static void drawShapes(Shape[] ss){
        for(int i = 0; i < ss.length; i++){
            ss[i].draw();
        }
    }

    public static void main(String[] args) {
        Random rand = new Random();
        Shape[] s = new Shape[9];

        for(int i = 0; i < s.length; i++){
            switch(rand.nextInt(3)) {
                case 0: s[i] = new Circle();break;
                case 1: s[i] = new Square();break;
                case 2: s[i] = new Triangle();break;
            }
        }

        drawShapes(s);
    }
}
```

► 运行结果：控制台输出(含字面量行)：

```
Calling Circle.draw()
Calling Circle.erase()
Calling Square.draw()
Calling Square.erase()
Calling Triangle.draw()
Calling Triangle.erase()
```

№54 TestPackage.java

Console

exp5_6

```
package chp5_advanced.exp5_6;

import chp5_advanced.exp5_6.graphics.twoD.Point;
import chp5_advanced.exp5_6.graphics.twoD.Rectangle;

public class TestPackage {
    public static void main(String args[]) {
        Point p = new Point(2, 3);
        Rectangle r = new Rectangle(p, 10, 10);
        System.out.println("The area of the rectangle is " + r.area());
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№55 ListTest.java

Console

exp5_7

```
package chp5_advanced.exp5_7;
import java.util.LinkedList;
import java.util.List;
public class ListTest {
    public static void main(String[] args) {
        // 注意：列表中只存放Integer 类型的对象!
        List listofInteger = new LinkedList();
        listofInteger.add(new Integer(2000));
        listofInteger.add("8");
        Integer x = (Integer) listofInteger.get(0);
        System.out.println(x);
        x = (Integer) listofInteger.get(1);
        System.out.println(x);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№56 ListTestWithGenerics.java

Console

exp5_8

```
package chp5_advanced.exp5_8;
import java.util.LinkedList;
import java.util.List;
public class ListTestWithGenerics {
    public static void main(String[] args) {
        List<Integer> listofInteger = new LinkedList<Integer>();
        listofInteger.add(new Integer(2000));
        listofInteger.add("8"); // Illegal
        // listofInteger.add(new Integer(8));
        // listofInteger.add(8);
        Integer x = listofInteger.get(0);
        System.out.println(x);
        x = listofInteger.get(1);
        System.out.println(x);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№57 MyBoxTest.java

Console

exp5_9

```
package chp5_advanced.exp5_9;
class MyBox<T> {
    private T t;
    public void add(T t) {
        this.t = t;
    }
    public T get() {
        return t;
    }
}
public class MyBoxTest{
    public static void main(String args[]){
        MyBox<Integer> aBox;
        aBox = new MyBox<Integer>();
        aBox.add(1000);
        Integer i = aBox.get();
        System.out.println("The Integer is : "+i);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp5_advanced.exp5_10;
import java.util.LinkedList;
class Cage<E> extends LinkedList<E>{};
class Animal{
    public void feedMe(){ }
}
class Lion extends Animal{
    public void feedMe(){
        System.out.println("Feeding lions");
    }
}
class ButterFly extends Animal{
    public void feedMe(){
        System.out.println("Feeding butterflies");
    }
}
public class WildcardsTest{
    public static void main(String args[]){
        WildcardsTest t = new WildcardsTest();
        Cage<Lion> lionCage = new Cage<Lion>();
        Cage<ButterFly> butterflyCage = new Cage<ButterFly>();
        lionCage.add(new Lion());
        butterflyCage.add(new ButterFly());
        t.feedAnimals(lionCage);
        t.feedAnimals(butterflyCage);
    }
    void feedAnimals(Cage<? extends Animal> someCage) {
        for (Animal a:someCage)
            a.feedMe();
    }
}
```

► 运行结果：控制台输出(含字面量行):
Feeding lions
Feeding butterflies

```
package chp5_advanced.exp5_11;
class MyBox<T> {
    private T t;
    public void add(T t) {
        this.t = t;
    }
    public T get() {
        return t;
    }
    public <U> void inspect(U u){
        System.out.println(" T: " + t.getClass().getName());
        System.out.println(" U: " + u.getClass().getName());
        System.out.println();
    }
}
public class BoxTest{
    public static void main(String[] args) {
        MyBox<Integer> integerBox = new MyBox<Integer>();
        integerBox.add(new Integer(10));
        System.out.println("The first inspection:");
        integerBox.inspect("some text");
        System.out.println("The second inspection:");
        integerBox.inspect(new Double(100.0));
    }
}
```

► 运行结果：控制台输出(含字面量行):
The first inspection:
The second inspection:

```
package chp5_advanced.exp5_12;

import java.util.HashSet;
import java.util.Set;

public class FindDups {
    public static void main(String args[]) {

        String[] words = {"java", "FindDups", "i", "came", "i", "saw", "i", "left"};

        Set<String> s = new HashSet<String>();

        for (int i=0; i<words.length; i++){
            if (!s.add(words[i]))
                System.out.println("Duplicate detected: "+words[i]);
        }

        System.out.println(s.size()+" distinct words detected: "+s);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№61 Deal.java Console

exp5_13

```
package chp5_advanced.exp5_13;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
class Deal {
    public static void main(String args[]) {
        int numHands = Integer.parseInt(args[0]);
        int cardsPerHand = Integer.parseInt(args[1]);

        // 生成一副牌（含52张牌）。
        String[] suit = new String[] {"spades", "hearts", "diamonds", "clubs"};
        String[] rank = new String[]
            {"ace", "2", "3", "4", "5", "6", "7", "8", "9", "10", "jack", "queen", "king"};
        List<String> deck = new ArrayList<String>();
        for (String ss : suit)
            for (String sr : rank)
                deck.add(sr + " of " + ss);

        Collections.shuffle(deck); //随机改变deck中元素的排列次序，即洗牌。

        for (int i=0; i<numHands; i++)
            System.out.println(dealHand(deck, cardsPerHand)); //生成一手牌并将其输出。
    }

    public static List dealHand(List<String> deck, int n) {
        int deckSize = deck.size();
        List<String> handView = deck.subList(deckSize-n, deckSize); //从deck中截取一个子链表。
        List<String> hand = new ArrayList<String>(handView); //利用该子链表创建一个链表。
        handView.clear(); //将子链表清空。
        return hand;
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№62 Counter.java Console

exp5_14

```
package chp5_advanced.exp5_14;
import java.util.LinkedList;
import java.util.Queue;
public class Counter {
    public static void main(String[] args){
        int time = 5;
        Queue<Integer> queue = new LinkedList<Integer>();
        for (int i = time; i >= 0; i--)
            queue.add(i);
        while (!queue.isEmpty()) {
            System.out.println(" "+queue.remove());
            try{
                Thread.sleep(1000);
            }catch(InterruptedException e){ }
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

№63 Freq.java Console

exp5_15

```
package chp5_advanced.exp5_15;

import java.util.Map;
import java.util.TreeMap;

public class Freq {

    public static void main(String args[]) {
        String[] words={"if", "it", "is", "to", "be", "it", "is", "up",
            "to", "me", "to", "delegate"};

        Integer freq ;
        Map<String, Integer> m = new TreeMap<String, Integer>();

        // 构造字符串数组words的单词频率表. 以单词为key, 以词频为value.
        for (String a : words) {
            freq = m.get(a); //获取指定单词的词频。

            //词频递增
            if (freq==null){
                freq = new Integer(1);
            }else{
                freq = new Integer(freq.intValue() + 1);
            }
            m.put(a, freq); //在Map中更改词频。
        }

        System.out.println(m.size() + " distinct words detected:");
        System.out.println(m);
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp5_advanced.exp5_16;
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;
public class ListIteratorDemo {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<Integer>();
        //向list中添加元素。
        for(int i=1; i<5; i++){
            list.add(new Integer(i));
        }
        System.out.println("The original list : "+list);
        ListIterator<Integer> listIter = list.listIterator(); //创建List的iterator。
        listIter.add(new Integer(0)); //在序号为0的元素前添加一个元素。
        System.out.println("After add at beginning:"+list);
        if ( listIter.hasNext() ) {
            int i = listIter.nextIndex(); //i的值将为1。
            listIter.next(); //返回序号为1的元素。
            listIter.set(new Integer(9)); //修改list中的序号为1的元素。
            System.out.println("After set at "+i+" : "+list);
        }
        if ( listIter.hasNext() ) {
            int i = listIter.nextIndex(); // i的值将为2。
            listIter.next();
            listIter.remove(); //删除序号为2的元素。
            System.out.println("After remove at "+i+" : "+list);
        }
    }
}
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

```
package chp5_advanced.exp5_18;
enum Week {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
public class EnumValuesTest{
    public static void main( String args[]){

        for (Week w:Week.values()){
            System.out.print ( w.name()+" " );
        }
        System.out.println ( );
    }
}
```

► 运行结果: 演示性代码 (类/接口定义或片段), 无独立 main 输出。

```
package chp5_advanced.exp5_19;
enum Coin2 {
    PENNY(1,CoinColor2.COPPER),
    NICKEL(5,CoinColor2.NICKEL),
    DIME(10,CoinColor2.SILVER),
    QUARTER(25,CoinColor2.SILVER);

    private final int value;
    private final CoinColor2 color;
    Coin2(int value, CoinColor2 color) {
        this.value = value;
        this.color = color;
    }
    public int value() { return value; }
    public CoinColor2 color(){ return color;}
}

enum CoinColor2 { COPPER, NICKEL, SILVER }

public class CoinTest2 {
    public static void main(String[] args) {

        // System.out.println(":"+Coin.PENNY.value());
        for (Coin2 c : Coin2.values())
            System.out.println(c + ": "+ c.value() +", " + c.color());
    }
}

/* enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);

    private final int value;
    Coin(int value) { this.value = value; }
    public int value() { return value; }
}

public class CoinTest {
    public static void main(String[] args) {

        // System.out.println(":"+Coin.PENNY.value());
        for (Coin c : Coin.values())
            System.out.println(c + ": "+ c.value() +"? " +
                color(c));
    }

    private enum CoinColor2 { COPPER, NICKEL, SILVER }

    private static CoinColor2 color(Coin c) {
        switch(c) {
            case PENNY:
                // System.out.println("in switch: "+c+" "+Coin.PENNY.value());
                return CoinColor2.COPPER;
            case NICKEL:
                return CoinColor2.NICKEL;
            case DIME: case QUARTER:
                return CoinColor2.SILVER;
            default:
                throw new AssertionError("Unknown coin: " + c);
        }
    }
}

*/
```

► 运行结果: 控制台按 main 逻辑输出 (含变量计算结果)。

```
package chp5_advanced.exp5_19;
enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);
    private final int value;
    Coin(int value) {
        this.value = value;
    }
    public int value() {
        return value;
    }
}
enum CoinColor { COPPER, NICKEL, SILVER }
public class CoinTest {
    public static void main(String[] args) {
        for (Coin c : Coin.values()){
            System.out.print(c + ": "+ c.value() +", ");
            switch(c) {
                case PENNY:
                    System.out.println(CoinColor.COPPER);
                    break;
                case NICKEL:
                    System.out.println(CoinColor.NICKEL);
                    break;
                case DIME:
                case QUARTER:
                    System.out.println(CoinColor.SILVER);
                    break;
            }
        }
    }
}
```

► 运行结果：控制台按 main 逻辑输出（含变量计算结果）。

```
package chp5_advanced.exp5_20;
public class AutoBoxingTest{
    public static void main(String args[]){
        Integer x, y;
        int c;
        x = 22;
        y = 15;
        if ( (c = x.compareTo(y)) == 0)
            System.out.println("x is equal to y");
        else
            if ( c < 0 )
                System.out.println("x is less than y");
            else
                System.out.println("x is greater than y");

        System.out.println("The sum of x and y is "+(x+y) );
    }
}
```

► 运行结果：控制台输出(含字面量行):

x is equal to y
x is less than y
x is greater than y

实验6 异常处理 (exp6)

№69 HelloWorld.java Console

exp6_1

```
package chp6_exception.exp6_1;

public class HelloWorld{

    public static void main(String args[ ]){
        int i=0;
        String greetings[ ]={ "Hello World!", "Hello!",
                                "HELLO WORLD!!!"};
        while ( i<4){
            System.out.println(greetings[i]);
            i++;
        }
        System.out.println("Normal ended.");
    }
}
```

► 运行结果：控制台输出(含字面量行):
Normal ended.

№70 TestListOfNumbers1.java FileIO

exp6_2

```
package chp6_exception.exp6_2;

import java.io.FileWriter;
import java.io.PrintWriter;
import java.util.ArrayList;

class ListOfNumbers {
    private ArrayList<Integer> list;
    private static final int size = 10;

    public ListOfNumbers () {
        list = new ArrayList<Integer>(size);
        for (int i = 0; i < size; i++)
            list.add(new Integer(i));
    }
    //将list保存到OutFile.txt文件中。
    public void writeList() {
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));

        for (int i = 0; i < size; i++)
            out.println("Value at: " + i + " = " + list.get(i));

        out.close();
    }
}

public class TestListOfNumbers1 {
    public static void main(String[] args) {
        ListOfNumbers list = new ListOfNumbers();
        list.writeList();
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台通常无输出。

```
package chp6_exception.exp6_3;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
class ListOfNumbers {
    private ArrayList<Integer> list;
    private static final int size = 10;
    public ListOfNumbers () {
        list = new ArrayList<Integer>(size);
        for (int i = 0; i < size; i++){
            list.add(new Integer(i));
        }
    }
    public void writeList() {
        PrintWriter out = null;
        try {
            System.out.println("Entering try statement");
            out = new PrintWriter(new FileWriter("OutFile.txt"));
            for (int i = 0; i < size; i++){
                out.println("Value at: " + i + " = " + list.get(i));
            }
        } catch (IOException e) { //处理越界异常。
            System.err.println("Caught IOException: " +
                e.getMessage());
        } catch (IOException e) { //处理I/O异常。
            System.err.println("Caught IOException: " + e.getMessage());
        } finally { //最后清理。
            if (out != null) {
                System.out.println("Closing PrintWriter");
                out.close();
            } else {
                System.out.println("PrintWriter not open");
            }
        }
    }
}
public class TestListOfNumbers {
    public static void main(String[] args) {
        ListOfNumbers list = new ListOfNumbers();
        list.writeList();
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台另打印: Entering try statement Closing PrintWriter PrintWriter not open。

```
package chp6_exception.exp6_4;
public class HelloWorld2{
    public static void main(String args[ ]){
        int i=0;
        String greetings[ ]={ "Hello World!", "Hello!",
            "HELLO WORLD!!" };
        while ( i<4){
            try {
                System.out.println(greetings[i]);
                i++;
            } catch (ArrayIndexOutOfBoundsException e){
                System.out.println("Caught ArrayIndexOutOfBoundsException");
                break;
            } finally{
                System.out.println("This is always printed");
            }
        }
    }
}
```

► 运行结果：控制台输出(含字面量行):
Caught ArrayIndexOutOfBoundsException
This is always printed

```
package chp6_exception.exp6_5;

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Vector;

class ListOfNumbersDeclared {
    private Vector<Integer> victor;
    private static final int size = 10;

    public ListOfNumbersDeclared () {
        victor = new Vector<Integer>(size);
        for (int i = 0; i < size; i++)
            victor.addElement(new Integer(i));
    }

    //声明抛出异常
    public void writeList() throws IOException, ArrayIndexOutOfBoundsException {
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));

        for (int i = 0; i < size; i++)
            out.println("Value at: " + i + " = " + victor.elementAt(i));

        out.close();
    }
}

public class TestOfDeclared {
    public static void main(String[] args) {
        try{
            ListOfNumbersDeclared list = new ListOfNumbersDeclared();
            list.writeList();
        }catch(Exception e){
            System.err.println("Caught Exception: " + e.getMessage()+"\n");
        }

        System.out.println("A list of numbers is created and stored in OutFile.txt");
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台另打印：A list of numbers is created and stored in OutFile.txt.

► 运行结果：演示性代码（类/接口定义或片段），无独立 main 输出。

```
package chp6_exception.exp6_7;

public class AssertionDemo {
    public static void main(String[] args) {
        int month = 13;
        switch (month) {
            case 1: System.out.println("January"); break;
            case 2: System.out.println("February"); break;
            case 3: System.out.println("March"); break;
            case 4: System.out.println("April"); break;
            case 5: System.out.println("May"); break;
            case 6: System.out.println("June"); break;
            case 7: System.out.println("July"); break;
            case 8: System.out.println("August"); break;
            case 9: System.out.println("September"); break;
            case 10: System.out.println("October"); break;
            case 11: System.out.println("November"); break;
            case 12: System.out.println("December"); break;
            default: System.out.println("false");
                    assert false:"Hey, that's not a valid month!"; break;
        }
    }
}
```

► 运行结果：控制台输出(含字面量行):

```
January
February
March
April
May
June
July
August
September
October
November
December
...
```

实验7 文件 IO (exp7)

Nº76 CopyBytes.javaFileIO

exp7_1

```
package chp7_io.exp7_1;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class CopyBytes {
    public static void main(String[] args) throws IOException {
        //创建两个File类对象。
        File outputFile = new File("src\\chp7_io\\exp7_1\\outagainb.txt");
        //创建文件输入/输出字节流。
        FileInputStream in = new FileInputStream("src\\chp7_io\\exp7_1\\farrago.txt");
        FileOutputStream out = new FileOutputStream(outputFile);
        int c;
        //读写文件流中的数据。
        while ((c = in.read()) != -1)
            out.write(c);
        //关闭流。
        in.close();
        out.close();
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台通常无输出。

Nº77 Copy.javaFileIO

exp7_2

```
package chp7_io.exp7_2;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class Copy {
    public static void main(String[] args) throws IOException {
        //创建文件字符输入/输出流。
        FileReader in = new FileReader("src\\chp7_io\\exp7_2\\farrago.txt");
        FileWriter out = new FileWriter("src\\chp7_io\\exp7_2\\outagainc.txt");
        int c;
        //读、写数据。
        while ((c = in.read()) != -1)
            out.write(c);
        //关闭流。
        in.close();
        out.close();
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台通常无输出。


```
package chp7_io.exp7_3;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintWriter;

public class ReverseThread extends Thread {
    private PrintWriter out = null;
    private BufferedReader in = null;

    public ReverseThread(PrintWriter out, BufferedReader in) {
        this.out = out;
        this.in = in;
    }

    //逆序线程的线程体。
    public void run() {
        if (out != null && in != null) {
            try {
                String input;
                while ((input = in.readLine()) != null) {
                    out.println(reverseIt(input));
                    out.flush();
                }
                out.close();
            } catch (IOException e) {
                System.err.println("ReverseThread run: " + e);
            }
        }
    }

    //实现单词的逆序。
    private String reverseIt(String source) {
        int i, len = source.length();
        StringBuffer dest = new StringBuffer(len);

        for (i = (len - 1); i >= 0; i--)
            dest.append(source.charAt(i));
        return dest.toString();
    }
}
```

► **运行结果：**文件操作：向目标文件写入内容；控制台通常无输出。

```
package chp7_io.exp7_3;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.io.PipedReader;
import java.io.PipedWriter;
import java.io.PrintWriter;
import java.io.Reader;

public class RhymingWords {
    public static void main(String[] args) throws IOException {

        FileReader words = new FileReader("src\\chp7_io\\exp7_3\\words.txt");

        // 进行单词的逆序、排序、再逆序还原。
        Reader rhymedWords = reverse(sort(reverse(words)));

        // 将处理后的单词列表输出显示。
        BufferedReader in = new BufferedReader(rhymedWords);
        String input;

        while ((input = in.readLine()) != null)
            System.out.println(input);
        in.close();
    }

    //单词逆序方法。
    public static Reader reverse(Reader source) throws IOException {

        BufferedReader in = new BufferedReader(source);

        PipedWriter pipeOut = new PipedWriter();
        PipedReader pipeIn = new PipedReader(pipeOut);
        PrintWriter out = new PrintWriter(pipeOut);

        new ReverseThread(out, in).start();

        return pipeIn;
    }

    //单词排序方法。
    public static Reader sort(Reader source) throws IOException {

        BufferedReader in = new BufferedReader(source);

        PipedWriter pipeOut = new PipedWriter();
        PipedReader pipeIn = new PipedReader(pipeOut);
        PrintWriter out = new PrintWriter(pipeOut);

        new SortThread(out, in).start();

        return pipeIn;
    }
}
```

► **运行结果：**文件操作：向目标文件写入内容；控制台通常无输出。

```
package chp7_io.exp7_3;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintWriter;

public class SortThread extends Thread {
    private PrintWriter out = null;
    private BufferedReader in = null;

    public SortThread(PrintWriter out, BufferedReader in) {
        this.out = out;
        this.in = in;
    }

    //排序线程的线程体。
    public void run() {
        int MAXWORDS = 50;

        if (out != null && in != null) {
            try {
                String[] listofWords = new String[MAXWORDS];
                int numwords = 0;

                while ((listofWords[numwords] = in.readLine()) != null)
                    numwords++;
                quicksort(listofWords, 0, numwords-1);
                for (int i = 0; i < numwords; i++)
                    out.println(listofWords[i]);
                out.close();
            } catch (IOException e) {
                System.err.println("SortThread run: " + e);
            }
        }
    }

    //实现快速排序算法。
    private static void quicksort(String[] a, int lo0, int hi0) {
        int lo = lo0;
        int hi = hi0;

        if (lo >= hi)
            return;

        String mid = a[(lo + hi) / 2];
        while (lo < hi) {
            while (lo < hi && a[lo].compareTo(mid) < 0)
                lo++;
            while (lo < hi && a[hi].compareTo(mid) > 0)
                hi--;
            if (lo < hi) {
                String T = a[lo];
                a[lo] = a[hi];
                a[hi] = T;
                lo++;
                hi--;
            }
        }
        if (hi < lo) {
            int T = hi;
            hi = lo;
            lo = T;
        }
        quicksort(a, lo0, lo);
        quicksort(a, lo == lo0 ? lo+1 : lo, hi0);
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台通常无输出。

```
package chp7_io.exp7_4;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class DataIOTest {
    public static void main(String[] args) throws IOException {
        // 创建数据输出流，前端套接文件流并以invoice1.txt为输出目的地。
        DataOutputStream out = new DataOutputStream(new
            FileOutputStream("src\\chp7_io\\exp7_4\\invoice1.txt"));

        //定义要保存的数据数组。
        double[] prices = { 19.99, 9.99, 15.99, 3.99, 4.99 };
        int[] units = { 12, 8, 13, 29, 50 };
        String[] desc = { "Java T-shirt", "Java Mug",
            "Duke Juggling Dolls",
            "Java Pin", "Java Key Chain" };

        //将prices,units以及desc中的数据以Tab键为分割符保存在文件中。
        for (int i = 0; i < prices.length; i++) {
            out.writeDouble(prices[i]);
            out.writeChar('\t');
            out.writeInt(units[i]);
            out.writeChar('\t');
            out.writeUTF(desc[i]);
            out.writeChar('\t');
        }
        out.close();
        // 创建数据输入流，将上面保存的文件再次打开并读取。
        DataInputStream in = new DataInputStream(new
            FileInputStream("src\\chp7_io\\exp7_4\\invoice1.txt"));

        double price;
        int unit;
        String desc;
        double total = 0.0;
        for (int i = 0; i < prices.length; i++) {
            price = in.readDouble();
            in.readChar(); // 扔掉tab键
            unit = in.readInt();
            in.readChar(); // 扔掉tab键
            desc = in.readUTF();
            in.readChar(); // 扔掉tab键

            System.out.println("You've ordered " +
                unit + " units of " +
                desc + " at $" + price);

            total = total + unit * price;
        }
        System.out.println("For a TOTAL of: $" + total);
        in.close();
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台另打印：You've ordered For a TOTAL of: \$。

```
package chp7_io.exp7_5;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class StandardIO {
    public static void main(String[] args) throws IOException {
        int i=0;
        boolean flag;
        String s;
        BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("请输入 (exit退出) : ");
        try {
            s = in.readLine();
            while (!s.equals("exit")) {
                try {
                    flag=false;
                    i = Integer.parseInt(s);
                } catch (NumberFormatException e) {
                    flag=true;
                    System.out.println("您刚输入的是: " + s);
                }
                if(flag==false){System.out.println("整数: " + i);}
                s = in.readLine();
            }
            System.out.println("输入完毕! ");
            in.close();
        } finally {
        }
    }
}
```

► 运行结果：文件操作：读取文件并处理，打印：请输入 (exit退出)：您刚输入的是：整数：。

```
package chp7_io.exp7_6;
import java.io.File;
import java.util.Date;
public class RenameFile{
    //显示文件基本信息。
    private static void fileData(File f) {
        System.out.println(
            "Absolute path: " + f.getAbsolutePath() +
            "\n Can read: " + f.canRead() +
            "\n Can write: " + f.canWrite() +
            "\n getName: " + f.getName() +
            "\n getParent: " + f.getParent() +
            "\n getPath: " + f.getPath() +
            "\n Length: " + f.length() +
            "\n lastModified: " + new Date(f.lastModified());
        if(f.isFile())
            System.out.println("It's a file");
        else if(f.isDirectory())
            System.out.println("It's a directory");
    }
    //将命令行第一个参数是原来的文件名，第二个参数是新文件名。
    public static void main(String[] args) {
        File old = new File(args[0]);
        File rname = new File(args[1]);
        System.out.println("The original file's information:");
        fileData(old);
        old.renameTo(rname);
        System.out.println("\n The file information after rename:");
        fileData(rname); //文件重命名。
        if (!old.exists()){
            System.out.println("\n The original file never exists.");
        }
    }
}
```

► 运行结果：控制台输出(含字面量行):

```
It's a file
It's a directory
The original file's information:
\n The file information after rename:
\n The original file never exists.
```

```
package chp7_io.exp7_7;
import java.io.RandomAccessFile;
public class RandomAccessTest{
    public static void main(String args[]) throws Exception{
        long filePoint = 0 ;
        String s;
        RandomAccessFile file = new RandomAccessFile("src\\chp7_io\\exp7_7\\RandomAccessTest.java","r");
        long fileLength = file.length();
        while (filePoint<fileLength){
            s = file.readLine();
            System.out.println(s);
            filePoint = file.getFilePointer();
        }
        file.close();
    }
}
```

► 运行结果：文件操作：读取文件并处理。

```
package chp7_io.exp7_8;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.ObjectOutputStream;
import java.util.Date;
public class SerializeDate {
    Date d ;
    SerializeDate( ){
        d= new Date( );
        try {
            FileOutputStream f = new FileOutputStream("date.ser");
            ObjectOutputStream s= new ObjectOutputStream(f);
            s.writeObject(d);
            f.close( );
        }catch( IOException e){
            e.printStackTrace( );
        }
    }
    public static void main(String args[]){
        SerializeDate b =new SerializeDate();
        System.out.println("The saved date is :"+b.d.toString());
    }
}
```

► 运行结果：文件操作：向目标文件写入内容；控制台另打印: The saved date is :。

```
package chp7_io.exp7_9;
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.util.Date;
public class UnSerializeDate{
    Date d = null ;
    UnSerializeDate(){
        try {
            FileInputStream f = new FileInputStream("src\\chp7_io\\exp7_9\\date.ser");
            ObjectInputStream s = new ObjectInputStream(f);
            d = (Date) s.readObject();
            f.close();
        }catch(Exception e){
            e.printStackTrace() ;
        }
    }
    public static void main(String args[]){
        UnSerializeDate a =new UnSerializeDate();
        System.out.println("The date read is :"+a.d.toString());
    }
}
```

► 运行结果：文件操作：读取文件并处理，打印: The date read is:。

```
package chp7_io.exp7_10;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.util.Date;
class Employee implements Serializable
{
    int id;
    String name;
    int age;
    int salary;
    String hireDay;
    String department;
    public Employee(){}
    public Employee(int id, String name, int age, int salary,
                    String hireDay, String department) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.salary = salary;
        this.hireDay = hireDay;
        this.department = department;
    }
    private void writeObject(ObjectOutputStream out) throws IOException {
        Date savedDate = new Date();

        out.writeInt(id);
        out.writeInt(age);
        out.writeUTF(name);
        out.writeInt(salary);
        out.writeUTF(hireDay);
        out.writeUTF(department);
        out.writeInt(savedDate.getYear());
    }
    private void readObject(ObjectInputStream in) throws IOException {
        Date readDate= new Date();
        int savedYear;
        id = in.readInt();
        age = in.readInt();
        name = in.readUTF();
        salary = in.readInt();
        hireDay = in.readUTF();
        department = in.readUTF();
        savedYear = in.readInt();
        age = age + (readDate.getYear()-savedYear);
    }
    public String toString(){
        return "id: " +id +"\n name: " + name + " \n age: "
            +age + "\n salary: " + salary + "\n hireDay: "
            + hireDay + "\n department: " + department ;
    }
}
public class ObjectSerial {
    public static void main(String args[]) throws IOException,
        ClassNotFoundException {
        Employee employ = new Employee(123456, "Tom", 23, 6000, "03/10/10", "intel");
        ObjectOutputStream fout1 = new ObjectOutputStream(
            new FileOutputStream("src\\chp7_io\\exp7_10\\data1.ser"));

        fout1.writeObject(employ) ; //使用Employee定制的writeObject方法
        fout1.close() ;
        employ = null;
        ObjectInputStream fin1 =new ObjectInputStream(
            new FileInputStream("src\\chp7_io\\exp7_10\\data1.ser"));

        employ = (Employee) fin1.readObject() ;//使用Employee定制的readObject方法
        fin1.close();
        System.out.println(employ.toString());
    }
}
```

► 运行结果: 文件操作: 向目标文件写入内容; 控制台通常无输出。

```
package chp7_io.exp7_11;
import java.io.Externalizable;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInput;
import java.io.ObjectInputStream;
import java.io.ObjectOutput;
import java.io.ObjectOutputStream;
import java.util.Date;
class EmployeeExtern{
    // some data that we will explicitly save
    int id;
    String name;
    int age;
    int salary;
    String hireDay;
```

```

String department;
public EmployeeExtern(){
EmployeeExtern(int id, String name, int age, int salary, String hireDay,
                String department) {

    this.id = id;
    this.name = name;
    this.age = age;
    this.salary = salary;
    this.hireDay = hireDay;
    this.department = department;
}

public String toString(){
    return "id: " + id + "\n name: " + name + " \n age: "
        + age + "\n salary: " + salary + "\n hireDay: "
        + hireDay + "\n department: " + department ;
}
}

class ManagerExtern extends EmployeeExtern implements Externalizable
{
    String position;
    public ManagerExtern(){
    ManagerExtern(int id, String name, int age, int salary, String hireDay,
                String department, String position)

    {
        super(id, name, age, salary, hireDay, department);
        this.position = position;
    }
}

//实现writeExternal () 方法
public void writeExternal(ObjectOutput out) throws IOException {
    Date savedDate = new Date();
    out.writeInt(id);
    out.writeInt(age);
    out.writeObject(name);
    out.writeInt(salary);
    out.writeObject(hireDay);
    out.writeObject(department);
    out.writeObject(position);
    out.writeInt(savedDate.getYear());
}

//实现readExternal () 方法
public void readExternal(ObjectInput in) throws IOException,
    java.Lang.ClassNotFoundException {

    Date readDate= new Date();
    int savedYear;
    id = in.readInt();
    age = in.readInt();
    name = (String) in.readObject();
    salary = in.readInt();
    hireDay = (String) in.readObject();
    department = (String) in.readObject();
    position = (String) in.readObject();
    savedYear = in.readInt();
    age = age + (readDate.getYear()-savedYear);
}

public String toString(){
    return super.toString()+ "\n position: " + position;
}
}

public class ObjectTest {
    public static void main(String args[]) {
        ManagerExtern manager = new ManagerExtern(456789, "Jack", 40, 10000, "80/10/10",
                                                    "intel", "teamLeader");

        try {
            ObjectOutputStream fout = new ObjectOutputStream(
                                                                    new FileOutputStream("src\\chp7_io\\exp7_11\\data2.ser"));

            fout.writeObject(manager);
            fout.close();
        } catch (Exception e) {
            System.out.println(e);
        }
        manager = null;
        try {
            ObjectInputStream fin = new ObjectInputStream(
                                                                    new FileInputStream("data2.ser"));
            manager = (ManagerExtern) fin.readObject();
        } catch (Exception e) {
            System.out.println(e);
        }
        System.out.println("manager "+manager.toString());
    }
}

```

► 运行结果：文件操作：向目标文件写入内容；控制台另打印: manager。


```
package chp7_io.exp7_12;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;

public class CopyBytesWithChannel {
    private static final int BSIZE = 1024; //定义缓冲区大小

    public static void main(String[] args) throws IOException {
        FileInputStream inStream = new FileInputStream("src\\chp7_io\\exp7_12\\farrago.txt");
        FileOutputStream outStream = new FileOutputStream("src\\chp7_io\\exp7_12\\outagain.txt");
        FileChannel inChannel = inStream.getChannel(); //获取该文件输入流的通道
        FileChannel outChannel = outStream.getChannel(); //获取该文件输出流的通道
        ByteBuffer buffer = ByteBuffer.allocate(BSIZE); //创建缓冲区
        while ((inChannel.read(buffer)) != -1) {
            buffer.flip(); // 使缓冲区准备好写操作
            outChannel.write(buffer);
            buffer.clear(); // 使缓冲区准备好读操作
        }
        inStream.close();
        outStream.close();
    }
}
```

► **运行结果：**文件操作：向目标文件写入内容；控制台通常无输出。

Nº90 HelloWorldSwing.java

GUI

exp8_1

```
package chp8_awt_swing.exp8_1;
import javax.swing.JFrame;
import javax.swing.JLabel;
//import java.awt.event.*;
public class HelloWorldSwing {
    public static void main(String[] args) {
        JFrame frame = new JFrame("HelloWorldSwing");
        final JLabel label = new JLabel("Hello World!");
        frame.getContentPane().add(label);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(2000,700);
        frame.setVisible(true);
    }
}
```

► 运行结果: 弹出窗口"HelloWorldSwing"; 显示文字:Hello World!。

Nº91 JFrameDemo.java

GUI

exp8_2

```
package chp8_awt_swing.exp8_2;

import java.awt.BorderLayout;
import java.awt.Dimension;

//import java.awt.event.*;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.UIManager;

public class JFrameDemo {
    public static void main(String s[]) throws Exception{

        //指定使用当前的Look&Feel装饰窗口。必须在创建窗口前设定。
        System.out.println(UIManager.getCrossPlatformLookAndFeelClassName());
        //UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        //UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");
        //UIManager.setLookAndFeel("com.sun.java.swing.plaf.windows.WindowsLookAndFeel");
        //UIManager.setLookAndFeel("com.sun.java.swing.plaf.mac.MacLookAndFeel");

        JFrame.setDefaultLookAndFeelDecorated(true);

        //创建并设定关闭窗口操作。
        JFrame frame = new JFrame("JFrameDemo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        //创建一个JLabel并加到窗口中。
        JLabel emptyLabel = new JLabel("xianzhe");
        emptyLabel.setPreferredSize(new Dimension(1750, 1000));
        frame.getContentPane().add(emptyLabel, BorderLayout.CENTER);

        //显示窗口。
        frame.pack();
        frame.setVisible(true);
    }
}
```

► 运行结果: 弹出窗口"JFrameDemo"; 显示文字:xianzhe。

Nº92 FrameWithPanel.java

GUI

exp8_3

```
package chp8_awt_swing.exp8_3;
import java.awt.Color;
import java.awt.GridLayout;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class FrameWithPanel extends JFrame {
    public static void main(String args[ ]) throws Exception{
        FrameWithPanel fr = new FrameWithPanel("Hello !");
        fr.setSize(200,200);
        fr.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel pan = new JPanel();
        pan.setSize(200,100);
        pan.setBackground(Color.yellow);
        pan.setLayout(new GridLayout(2,1));
        pan.add(new JButton("确定"));
        fr.setContentPane(pan);
        fr.setVisible(true);
    }

    public FrameWithPanel(String str){
        super(str);
    }
}
```

► 运行结果: 弹出窗口"窗口"; 按钮:确定。

```
package chp8_awt_swing.exp8_4;

import java.awt.FlowLayout;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class FlowLayoutWindow extends JFrame {

    public FlowLayoutWindow() {
        setLayout(new FlowLayout());
        add(new JLabel("Buttons:"));
        add(new JButton("Button 1"));
        add(new JButton("2"));
        add(new JButton("Button 3"));
        add(new JButton("Long-Named Button 4"));
        add(new JButton("Button 5"));
    }

    public static void main(String args[]) {
        FlowLayoutWindow window = new FlowLayoutWindow();
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.setTitle("FlowLayoutWindow Application");
        window.pack(); //窗口的大小设置为适合组件最佳尺寸与布局所需的空间。
        window.setVisible(true);
    }
}
```

► 运行结果：弹出窗口"FlowLayoutWindow Application"；显示文字:Buttons；按钮:Button 1 / 2 / Button 3 / Long-Named Button 4 / Button 5。

```
package chp8_awt_swing.exp8_5;

import java.awt.BorderLayout;

import javax.swing.JButton;
import javax.swing.JFrame;

public class BorderLayoutWindow extends JFrame {

    public BorderLayoutWindow() {
        setLayout(new BorderLayout());

        add(new JButton("North"), "North");
        add( new JButton("South"),"South");
        add( new JButton("East"),"East");
        add( new JButton("West"),"West");
        add( new JButton("Center"),"Center");
    }

    public static void main(String args[]) {
        BorderLayoutWindow window = new BorderLayoutWindow();

        window.setTitle("BorderWindow Application");
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.pack();
        window.setVisible(true);
    }
}
```

► 运行结果：弹出窗口"BorderWindow Application"；按钮:North / South / East / West / Center。

```
package chp8_awt_swing.exp8_6;

import java.awt.GridLayout;

import javax.swing.JButton;
import javax.swing.JFrame;

public class GridLayoutWindow extends JFrame {

    public GridLayoutWindow() {
        setLayout(new GridLayout(0,2));

        add(new JButton("Button 1"));
        add(new JButton("2"));
        add(new JButton("Button 3"));
        add(new JButton("Long-Named Button 4"));
        add(new JButton("Button 5"));
        add(new JButton("6"));
    }

    public static void main(String args[]) {
        GridLayoutWindow window = new GridLayoutWindow();

        window.setTitle("GridWindow Application");
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.pack();
        window.setVisible(true);
    }
}
```

► 运行结果: 弹出窗口"GridWindow Application"; 按钮:Button 1 / 2 / Button 3 / Long-Named Button 4 / Button 5 / 6。

```
package chp8_awt_swing.exp8_7;

import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class CardLayoutWindow extends JFrame implements ActionListener {

    JPanel cards;
    CardLayout CLayout=new CardLayout();

    public CardLayoutWindow() {
        setLayout(new BorderLayout()); //设置Frame为BorderLayout。

        //创建摆放“卡片切换”按钮的panel,并添加到Frame中。
        JPanel cp = new JPanel();
        JButton bt = new JButton("卡片切换");
        bt.addActionListener(this);
        cp.add(bt);
        add("North", cp);

        //创建承放多个卡片的Panel, 设置为CardLayout。
        cards = new JPanel();
        cards.setLayout(CLayout);

        //创建cards中的第一个panel及其组件。
        JPanel p1 = new JPanel();
        p1.add(new JButton("Button 1"));
        p1.add(new JButton("Button 2"));
        p1.add(new JButton("Button 3"));

        //创建cards中的另一个panel及其组件。
        JPanel p2 = new JPanel();
        p2.add(new JTextField("TextField", 20));

        //把上述两个panel加到cards中。
        cards.add("Panel with Buttons", p1);
        cards.add("Panel with TextField", p2);

        //将cards放入Frame中。
        add("Center", cards);
    }

    //响应点击切换卡片按钮的事件。
    public void actionPerformed(ActionEvent e){
        CLayout.next(cards); //显示下一张卡片。
    }

    public static void main(String args[]) {
        CardLayoutWindow window = new CardLayoutWindow();

        window.setTitle("CardWindow Application");
        window.pack();
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.setVisible(true);
    }
}
```

► 运行结果: 弹出窗口"CardWindow Application"; 按钮:卡片切换 / Button 1 / Button 2 / Button 3。

```
package chp8_awt_swing.exp8_8;

import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;

import javax.swing.JButton;
import javax.swing.JFrame;

public class GridBagLayoutWindow extends JFrame {

    /**创建标签为name的button，并使用约束对象c将
    该button加入到采用gridbag布局管理器中。*/
    protected void makebutton(String name,
                               GridBagLayout gridbag,
                               GridBagConstraints c) {
        JButton button = new JButton(name);
        gridbag.setConstraints(button, c);
        add(button);
    }

    public GridBagLayoutWindow() {
        GridBagLayout gridbag = new GridBagLayout();
        GridBagConstraints c = new GridBagConstraints();

        setLayout(gridbag);

        c.fill = GridBagConstraints.BOTH; //组件充满显示区域。
        c.weightx = 1.0;
        makebutton("Button1", gridbag, c);
        makebutton("Button2", gridbag, c);
        makebutton("Button3", gridbag, c);

        c.gridwidth = GridBagConstraints.REMAINDER; //到行结束。
        makebutton("Button4", gridbag, c);

        c.weightx = 0.0; //恢复为缺省值。
        makebutton("Button5", gridbag, c); //另外一行，c.gridwidth同Button4。

        c.gridwidth = GridBagConstraints.RELATIVE; //所在行的倒数第二个组件。
        makebutton("Button6", gridbag, c);

        c.gridwidth = GridBagConstraints.REMAINDER; //到行结束。
        makebutton("Button7", gridbag, c);

        c.gridwidth = 1; //恢复为缺省值。
        c.gridheight = 2;
        c.weighty = 1.0;
        makebutton("Button8", gridbag, c);

        c.weighty = 0.0; //恢复为缺省值。
        c.gridwidth = GridBagConstraints.REMAINDER; //到行结束。
        c.gridheight = 1; //恢复为缺省值。
        makebutton("Button9", gridbag, c);
        makebutton("Button10", gridbag, c);
    }

    public static void main(String args[]) {
        GridBagLayoutWindow window = new GridBagLayoutWindow();

        window.setTitle("GridBagLayoutWindow Application");
        window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        window.pack();
        window.setVisible(true);
    }
}
```

► 运行结果：弹出窗口"GridBagLayoutWindow Application"。

```
package chp8_awt_swing.exp8_9;

import java.awt.Component;
import java.awt.Container;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;

public class BoxWindow extends JFrame {

    public BoxWindow() {
        Container contentPane = getContentPane();
        contentPane.setLayout(new BoxLayout(contentPane,
            BoxLayout.Y_AXIS));

        addAButton("Button 1", contentPane);
        addAButton("2", contentPane);
        addAButton("Button 3", contentPane);
        addAButton("Long-Named Button 4", contentPane);
        addAButton("Button 5", contentPane);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    private void addAButton(String text, Container container) {
        JButton button = new JButton(text);
        button.setAlignmentX(Component.CENTER_ALIGNMENT);
        container.add(button);
    }

    public static void main(String args[]) {
        BoxWindow window = new BoxWindow();

        window.setTitle("BoxLayout");
        window.pack();
        window.setVisible(true);
    }
}
```

► 运行结果：弹出窗口"BoxLayout"。

```
package chp8_awt_swing.exp8_10;
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
public class TestButton{
    public static void main(String args[] ){
        JFrame f = new JFrame("Test");
        f.setSize(200,100);
        f.setLayout(new FlowLayout(FlowLayout.CENTER));
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JButton b = new JButton("Press Me!");
        b.addActionListener(new ButtonHandler( )); //注册点击鼠标事件监听器。
        f.add(b);
        f.setVisible(true) ;
    }
}
//定义ActionEvent监听器类
class ButtonHandler implements ActionListener{
    public void actionPerformed(ActionEvent e){
        System.out.println("Action occurred");
        System.out.println("Button's label is:"+
            e.getActionCommand());
    }
}
```

► 运行结果：弹出窗口"Test"；按钮:Press Me!。

```
package chp8_awt.swing.exp8_11;
import java.awt.BorderLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;
import java.awt.event.MouseMotionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextArea;
public class ComplexListener implements
    MouseMotionListener, MouseListener, ActionListener{
    JFrame f;
    JTextArea tf;
    JButton bt;
    int number=1;
    public ComplexListener(){
        JLabel label=new JLabel("click and Drag the mouse");
        f=new JFrame("Complex Listener");
        tf=new JTextArea();
        bt = new JButton("退出");

        tf.addMouseMotionListener(this);
        tf.addMouseListener(this);
        bt.addActionListener(this);

        f.add(label, BorderLayout.NORTH);
        f.add(tf, BorderLayout.CENTER);
        f.add(bt, BorderLayout.SOUTH);
        f.setSize(300,200);
        f.setVisible(true);
    }
    // MouseMotionListener 的方法。
    public void mouseDragged(MouseEvent e){
        String s = number+++ " "+"The mouse Dragged: x= "+e.getX()+" y="+e.getY()+"\n";
        tf.append(s);
    }
    //MouseListener 的方法。
    public void mouseEntered(MouseEvent e){
        String s = number+++ " "+"The mouse entered"+" \n";
        tf.append(s);
    }
    public void mouseClicked(MouseEvent e){
        String s = number+++ " "+"The mouse cliked."+" \n";
        tf.append(s);
    }
    public void mouseExited(MouseEvent e){
        String s =number+++ " "+"The mouse exit."+" \n";
        tf.append(s);
    }
    // 未使用的 MouseMotionListener 方法。
    public void mouseMoved(MouseEvent e){ }
    // 未使用的 MouseListener 方法。
    public void mouseReleased(MouseEvent e){ }
    public void mousePressed(MouseEvent e){ }
    //ActionListener的方法。
    public void actionPerformed(ActionEvent e){
        System.exit(0);
    }
    public static void main(String args[]){
        ComplexListener two = new ComplexListener( );
    }
}
```

► 运行结果：弹出窗口"Complex Listener"；显示文字:click and Drag the mouse；按钮:退出。


```
package chp8_awt.swing.exp8_12;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextArea;

public class MultiListener extends JFrame implements ActionListener {
    JTextArea topTextArea;
    JTextArea bottomTextArea;
    JButton button1, button2;

    public MultiListener(String s) {
        super( s);
        JLabel l = null;
        GridBagLayout gridbag = new GridBagLayout();
        GridBagConstraints c = new GridBagConstraints();
        setLayout(gridbag);    //Frame设置为GridBagLayout布局管理器。
        c.fill = GridBagConstraints.BOTH;
        c.gridwidth = GridBagConstraints.REMAINDER;
        l = new JLabel("监听器听到的:");
        gridbag.setConstraints(l, c);
        add(l);
        c.weighty = 1.0;
        topTextArea = new JTextArea(5, 20);
        topTextArea.setEditable(false);
        gridbag.setConstraints(topTextArea, c);
        add(topTextArea);
        c.weightx = 0.0;
        c.weighty = 0.0;
        l = new JLabel("偷听者听到的:");
        gridbag.setConstraints(l, c);
        add(l);
        c.weighty = 1.0;
        bottomTextArea = new JTextArea(5, 20);
        bottomTextArea.setEditable(false);
        gridbag.setConstraints(bottomTextArea, c);
        add(bottomTextArea);
        c.weightx = 1.0;
        c.weighty = 0.0;
        c.gridwidth = 1;
        c.insets = new Insets(10, 10, 0, 10);
        button1 = new JButton("啦 啦 啦");
        gridbag.setConstraints(button1, c);
        add(button1);
        c.gridwidth = GridBagConstraints.REMAINDER;
        button2 = new JButton("你别说话!");
        gridbag.setConstraints(button2, c);
        add(button2);
        //当前MultiListener对象同时监听两个Button的事件。
        button1.addActionListener(this);
        button2.addActionListener(this);
        //为第二个Button再注册一个监听器。
        button2.addActionListener(new Eavesdropper());
        //向窗口注册响应关闭窗口操作的监听器。
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
        pack();
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        topTextArea.append(e.getActionCommand() + "\n");
    }

    //第二个Button的监听器类。
    class Eavesdropper implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            bottomTextArea.append("OK,"+e.getActionCommand() + "\n");
        }
    }

    public static void main(String[] args){
        MultiListener m=new MultiListener("Multilistener example" );
    }
}
```

► 运行结果：弹出口口"窗口"；显示文字:监听器听到的: / 偷听者听到的:；按钮:啦 啦 啦 / 你别说话!。

```
package chp8_awt_swing.exp8_13;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.JFrame;
public class MyFrameCanExit2 extends JFrame {
    public static void main(String args[ ]){
        MyFrameCanExit2 fr = new MyFrameCanExit2("Hello !");
        fr.addWindowListener( fr.new WindowHandler()); //注册窗口事件监听器。
        fr.setSize(200,200);
        fr.setVisible(true);
    }

    public MyFrameCanExit2(String str){
        super(str);
    }
    class WindowHandler extends WindowAdapter{
        public void windowClosing(WindowEvent e) {
            System.out.println(e.getSource());
            System.exit(0);
        }
    }
}
```

► 运行结果：弹出窗口"窗口"。

```
package chp8_awt_swing.exp8_14;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.JFrame;
public class MyFrameCanExit3 extends JFrame {
    public static void main(String args[ ]){
        MyFrameCanExit3 fr = new MyFrameCanExit3("Hello !");
        fr.addWindowListener( new WindowAdapter(){
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        }); //注册窗口事件监听器。
        fr.setSize(200,200);
        fr.setVisible(true);
    }

    public MyFrameCanExit3(String str){
        super(str);
    }
}
```

► 运行结果：弹出窗口"窗口"。

```
package chp8_awt.swing.exp8_15;

import java.awt.BorderLayout;
import java.awt.Component;
import java.awt.GridLayout;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTabbedPane;

public class TabbedPaneDemo extends JPanel {
    public TabbedPaneDemo() {
        ImageIcon icon = new ImageIcon("src\\chp8_awt_swing\\exp8_15\\new.gif");
        JTabbedPane tabbedPane = new JTabbedPane();

        Component panel1 = makeTextPanel("你好! ");
        tabbedPane.addTab("One", icon, panel1, "Does nothing");
        tabbedPane.setSelectedIndex(0);

        Component panel2 = makeTextPanel("Blah blah");
        tabbedPane.addTab("Two", icon, panel2, "Does twice as much nothing");

        Component panel3 = makeTextPanel("Blah blah blah");
        tabbedPane.addTab("Three", icon, panel3, "Still does nothing");

        Component panel4 = makeTextPanel("Blah blah blah blah");
        tabbedPane.addTab("Four", icon, panel4, "Does nothing at all");

        //将标签面板添加到当前面板中。
        setLayout(new GridLayout(1, 1));
        add(tabbedPane);
    }

    protected Component makeTextPanel(String text) {
        JPanel panel = new JPanel(false);
        JLabel filler = new JLabel(text);
        filler.setHorizontalAlignment(JLabel.CENTER);
        panel.setLayout(new GridLayout(1, 1));
        panel.add(filler);
        return panel;
    }

    public static void main(String[] args) {
        JFrame.setDefaultLookAndFeelDecorated(true);
        JFrame frame = new JFrame("TabbedPaneDemo");
        frame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {System.exit(0);}
        });

        frame.getContentPane().add(new TabbedPaneDemo(),
                                   BorderLayout.CENTER);

        frame.setSize(400, 125);
        frame.setVisible(true);
    }
}
```

► 运行结果：弹出窗口"TabbedPaneDemo"。

```
package chp8_awt.swing.exp8_16;

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.table.AbstractTableModel;

public class TableDemo extends JFrame {
    private boolean DEBUG = true;

    public TableDemo() {
        super("TableDemo");

        MyTableModel myModel = new MyTableModel();
        JTable table = new JTable(myModel);
        table.setPreferredScrollableViewportSize(new Dimension(500, 70));

        //表格常常放在滚动面板中。
        JScrollPane scrollPane = new JScrollPane(table);

        //将滚动面板放在窗口中。
        getContentPane().add(scrollPane, BorderLayout.CENTER);
    }
}
```

```

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});
}

/* 自定义表格数据模型。其中的方法都是AbstractTableModel中的方法。
   该类是内部类。*/
class MyTableModel extends AbstractTableModel {

    //定义表格的列名。
    final String[] columnNames = {"姓名", "学号", "专业", "性别", "年龄", "婚否"};

    //定义表格中的5行数据。
    final Object[][] data = {
        {"张山", "200101", "男",
         "计算机", new Integer(20), new Boolean(false)},
        {"王大民", "200103", "男",
         "计算机", new Integer(29), new Boolean(true)},
        {"刘均", "200204", "女",
         "电子工程", new Integer(21), new Boolean(false)},
        {"李力", "200208", "女",
         "自动控制", new Integer(25), new Boolean(true)},
        {"张爱军", "200301", "男",
         "机械制造", new Integer(31), new Boolean(false)}
    };

    public int getColumnCount() {
        return columnNames.length;
    }

    public int getRowCount() {
        return data.length;
    }

    public String getColumnName(int col) {
        return columnNames[col];
    }

    public Object getValueAt(int row, int col) {
        return data[row][col];
    }

    //获取列的数据类型，JTable使用该方法确定数据的显示格式。
    public Class getColumnClass(int c) {
        return getValueAt(0, c).getClass();
    }

    //如果表格是不可编辑的，则不需要实现该方法。
    public boolean isCellEditable(int row, int col) {
        //表格前两列不可编辑。
        if (col < 2) {
            return false;
        } else {
            return true;
        }
    }

    //改变网格单元的值。
    public void setValueAt(Object value, int row, int col) {

        //打印输出修改数据的位置。
        if (DEBUG) {
            System.out.println("Setting value at " + row + ", " + col
                               + " to " + value
                               + " (an instance of "
                               + value.getClass() + ")");
        }

        if (data[0][col] instanceof Integer
            && !(value instanceof Integer)) {
            try {
                data[row][col] = new Integer(value.toString());
                fireTableCellUpdated(row, col);
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(TableDemo.this,
                    "The \"" + getColumnName(col)
                    + "\" column accepts only integer values.");
            }
        } else {
            data[row][col] = value;
            fireTableCellUpdated(row, col);
        }
    }

    //打印输出修改后表格中的数据。
    if (DEBUG) {
        System.out.println("New value of data:");
        int numRows = getRowCount();
        int numCols = getColumnCount();

        for (int i=0; i < numRows; i++) {
            System.out.print("   row " + i + ":");
            for (int j=0; j < numCols; j++) {
                System.out.print(" " + data[i][j]);
            }
        }
    }
}

```

```

        System.out.println();
    }
    System.out.println("-----");
}
}

public static void main(String[] args) {
    TableDemo frame = new TableDemo();
    frame.pack();
    frame.setVisible(true);
}
}

```

► 运行结果：弹出窗口“窗口”；点击弹窗:The %。

№106 TreeDemo.java GUI

exp8_17

```

package chp8_awt_swing.exp8_17;

import java.awt.BorderLayout;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import javax.swing.JFrame;
import javax.swing.JScrollPane;
import javax.swing.JTree;
import javax.swing.event.TreeSelectionEvent;
import javax.swing.event.TreeSelectionListener;
import javax.swing.tree.DefaultMutableTreeNode;
import javax.swing.tree.TreeSelectionModel;

public class TreeDemo extends JFrame {

    //树中要显示的数据。
    String[][] data = {{"Books for Java Programmers",
                        "The Java Tutorial: Object-Oriented Programming for the Internet",
                        "The Java Tutorial Continued: The Rest of the JDK",
                        "The JFC Swing Tutorial: A Guide to Constructing GUIs",
                        "The Java Programming Language",
                        "The Java FAQ",
                        "The Java Class Libraries: An Annotated Reference",
                        "Concurrent Programming in Java: Design Principles and Patterns"},
                      {"Books for Java Implementers",
                        "The Java Virtual Machine Specification",
                        "The Java Language Specification"}
    };

    public TreeDemo() {
        super("TreeDemo");

        //创建树的根节点。
        DefaultMutableTreeNode top = new DefaultMutableTreeNode("The Java Series");

        //创建树的两个枝并添加到根节点上。
        for( int i = 0; i<data.length; i++){
            top.add(createBranch(data[i]));
        }

        //创建树，一次只允许选取一个节点。
        final JTree tree = new JTree(top);
        tree.getSelectionModel().setSelectionMode(
            TreeSelectionModel.SINGLE_TREE_SELECTION);

        //为树注册监听器，监听用户所选取节点的变化。
        tree.addTreeSelectionListener(new TreeSelectionListener() {
            public void valueChanged(TreeSelectionEvent e) {
                DefaultMutableTreeNode node = (DefaultMutableTreeNode)
                    tree.getLastSelectedPathComponent();
                if (node == null) return;
                Object nodeInfo = node.getUserObject();
                System.out.println(nodeInfo.toString());
            }
        });

        //创建滚动面板并将树放入其中。
        JScrollPane treeView = new JScrollPane(tree);

        //将滚动面板加到当前窗口中。
        getContentPane().add(treeView, BorderLayout.CENTER);
    }

    //创建以data[0]对象为根的一个枝。
    DefaultMutableTreeNode createBranch(String[] data){
        DefaultMutableTreeNode category = new DefaultMutableTreeNode(data[0]);
        DefaultMutableTreeNode book = null;

        for( int i =1; i<data.length; i++){
            book = new DefaultMutableTreeNode(data[i]);
            category.add(book);
        }

        return category;
    }
}

```

```
}

public static void main(String[] args) {
    JFrame frame = new TreeDemo();

    frame.addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });

    frame.setSize(400,250);
    frame.setVisible(true);
}
}
```

► 运行结果：弹出窗口“窗口”。

№107 Demo8_18.java GUI

exp8_18

```
package chp8_awt_swing.exp8_18;

import java.awt.Color;
import java.awt.Graphics;

import javax.swing.JFrame;
import javax.swing.JPanel;

class MyPanel extends JPanel{
    public void paint(Graphics g){
        setBackground(Color.ORANGE);
        super.paint(g);
        g.setColor(Color.cyan);
        g.fill3DRect(50, 50, 200, 90, false);
        g.setColor(Color.red);
        g.drawOval(300, 300, 100, 50);
    }
}

public class Demo8_18 {
    public static void main(String[] args) {
        JFrame jf=new JFrame("我是画笔");
        jf.add(new MyPanel());
        jf.setSize(500, 500);
        jf.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        jf.setVisible(true);
    }
}
```

► 运行结果：弹出窗口“我是画笔”；面板上用Graphics绘制图形。

```

package chp12_JDBC;

import java.sql.*;

import java.util.ArrayList;
import java.util.List;

public class JDBCExp {
    private static final String DRIVERCLASS = "com.microsoft.sqlserver.jdbc.SQLServerDriver";
    private final String url = "jdbc:sqlserver://127.0.0.1:1433;DatabaseName=msdb";
    private final String username = "sa";
    private final String password = "";
    private Connection conn;
    private Statement stmt;
    private PreparedStatement prpdStmt;
    private CallableStatement cablStmt;

    static {
        try {
            Class.forName(DRIVERCLASS);
        } catch (ClassNotFoundException e) {
            System.out.println("在加载数据库驱动时抛出异常，内容如下：");
            e.printStackTrace();
        }
    }

    // Statement
    public void useStatement(String sql) throws SQLException {
        stmt = conn.createStatement();
        stmt.executeUpdate(sql);
        stmt.close();
    }

    // PreparedStatement
    public void UsePreparedStatement(int[] id, String[] names) throws SQLException {
        prpdStmt = conn.prepareStatement("INSERT INTO Student VALUES(?,?)");// 预处理动态INSERT语句
        prpdStmt.clearBatch();// 清除Batch
        for (int i = 0; i < id.length; i++) {
            prpdStmt.setInt(1, id[i]);// 为动态SQL语句赋值
            prpdStmt.setString(2, names[i]);
            prpdStmt.addBatch();// 向Batch中添加INSERT语句
        }
        prpdStmt.executeBatch();// 批量执行Batch中的INSERT语句
        prpdStmt.close();
    }

    // CallableStatement
    public void UseCallableStatement(int[] id, String[] names) throws SQLException {
        cablStmt = conn.prepareCall("{call proc_student_insert(?,?)}");
        cablStmt.clearBatch();
        for (int i = 0; i < id.length; i++) {
            cablStmt.setInt(1, id[i]);
            cablStmt.setString(2, names[i]);
            cablStmt.addBatch();
        }
        cablStmt.executeBatch();
        cablStmt.close();
    }

    // 查询所有记录
    public void selectAll() {
        List<Object[]> list = new ArrayList<Object[]>();
        try {
            stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM Student");
            while (rs.next()) {
                Object note[] = new Object[2];
                for (int i = 0; i < note.length; i++) {
                    note[i] = rs.getObject(i + 1);
                }
                list.add(note);
            }
            stmt.close();
        } catch (SQLException e) {
            System.out.println("在查询记录时抛出异常，内容如下：");
            e.printStackTrace();
        }
        for (int i = 0; i < list.size(); i++) {
            System.out.println(list.get(i)[0].toString() + '\t'
                + list.get(i)[1].toString());
        }
    }

    public static void main(String[] args) throws SQLException {
        JDBCExp a = new JDBCExp();
        a.conn = DriverManager.getConnection(a.url, a.username, a.password);//建立连接

        //用普通语句创建数据表
        try{a.useStatement("DROP TABLE Student");}catch(SQLException e){}
        a.useStatement("CREATE TABLE Student(st_no int NOT Null PRIMARY KEY, st_name varchar(20) NOT Null)");

        //用预编译语句批添加数据
    }
}

```

```
int[] id={1,2,3};
String[] name={"张三","李四","王五"};
a.UsePreparedStatement(id,name);

//用存储过程批添加数据
try{a.useStatement("DROP PROCEDURE proc_student_insert");}catch(SQLException e){}
a.useStatement("CREATE PROCEDURE proc_student_insert @id int,@name varchar(20) AS "+
               "INSERT INTO Student VALUES(@id,@name)");

int[] id2={4,5};
String[] name2={"Amy","Joan"};
a.UseCallableStatement(id2,name2);

//查询记录
a.selectAll();

a.conn.close();//断开连接
}
}
```

► **运行结果：**连接数据库执行SQL；增删改无控制台输出(改数据库表)，查询则遍历ResultSet打印各行。


```
package chp12_JDBC.exp12_1;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JdbcTest {
    public static void main(String args[]) {

        String url= "jdbc:mysql://localhost/test";
        Connection con;
        String sql;
        Statement stmt;
        String num,name,sex;
        int age,math,eng,spec;

        try {

            Class.forName("com.mysql.jdbc.Driver");
        } catch (java.lang.ClassNotFoundException e) {
            System.err.print("ClassNotFoundException: ");
            System.err.println(e.getMessage());
        }

        try {
            con = DriverManager.getConnection(url, "root", "java"); //
            stmt = con.createStatement();

            //向学生表中插入一行记录。
            sql = "INSERT INTO STUDENT " +
                "VALUES('200108','赵小龙','男',20,71,62,76)";
            stmt.executeUpdate(sql);

            //检索student表中的所有记录并获取数据输出。
            sql = " SELECT * FROM STUDENT";
            ResultSet rs = stmt.executeQuery(sql);
            System.out.println("学号      姓名      性别      年龄"+
                "      高等数学  英语  专业课");

            while(rs.next()){
                num = rs.getString(1);
                name = rs.getString(2);
                sex = rs.getString(3);
                age = rs.getInt(4);
                math = rs.getInt(5);
                eng = rs.getInt("英语");
                spec = rs.getInt("专业课");
                System.out.println(num+"      "+name+"      "+sex+"      "+age+"      "+math
                    +"      "+eng+"      "+spec);
            }

            //检索高等数学成绩80分以上的学生信息。
            rs = stmt.executeQuery("SELECT 学号,姓名,高等数学,英语,专业课 "+
                "FROM STUDENT "+
                "WHERE 高等数学>=80" );

            System.out.println();
            System.out.println("The students whose math mark is beyond 80 are:");
            while(rs.next()){
                num = rs.getString(1);
                name = rs.getString(2);
                math = rs.getInt(3);
                eng = rs.getInt("英语");
                spec = rs.getInt("专业课");
                System.out.println("学号="+num+"      "+"姓名="+name+"      "+"高等数学="+
                    math+"      "+"英语="+eng+"      "+"专业课="+spec);
            }

            //关闭连接。
            stmt.close();
            con.close();
        } catch (SQLException ex) {
            System.err.println("SQLException: " + ex.getMessage());
        }
    }
}
```

► 运行结果：连接数据库执行SQL；增删改无控制台输出(改数据库表)，查询则遍历ResultSet打印各行。

```

package chp12_JDBC.exp12_2;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class PreparedTest {
    public static void main(String args[]) {
        String url = "jdbc:mysql://localhost/test";
        Connection con;
        Statement stmt;
        String name;
        int sale;

        try {
            Class.forName("com.mysql.jdbc.Driver");
        } catch (java.lang.ClassNotFoundException e) {
            System.err.print("ClassNotFoundException: ");
            System.err.println(e.getMessage());
        }

        try {
            con = DriverManager.getConnection(url, "root", "java");
            stmt = con.createStatement();
            showValues(stmt, "The original values:");
            PreparedStatement updateSales;
            String updateString = "update COFFEES " +
                "set SALES = ? where COF_NAME = ? ";
            updateSales = con.prepareStatement(updateString);
            int [] salesForWeek = {150, 140, 50, 145, 85};
            String [] coffees = {"Colombian", "French_Roast", "Espresso",
                "Colombian_Decaf", "French_Roast_Decaf"};

            int len = coffees.length;
            for(int i = 0; i < len; i++) {
                updateSales.setInt(1, salesForWeek[i]);
                updateSales.setString(2, coffees[i]);
                updateSales.executeUpdate();
            }
            System.out.println();
            showValues(stmt, "The new values :");
        } catch (SQLException ex) {
            System.err.println("SQLException: " + ex.getMessage());
        }
    }

    public static void showValues(Statement stmt, String s) throws SQLException{
        String name;
        int sale;
        ResultSet rs = stmt.executeQuery("SELECT COF_NAME, SALES "+
                                           "FROM COFFEES ");

        System.out.println(s);
        while(rs.next()){
            name = rs.getString(1);
            sale = rs.getInt(2);
            System.out.println( "COFF_NAME = "+name+" ;    "+ "SALE = "+sale);
        }
    }
}

```

► 运行结果：连接数据库执行SQL；增删改无控制台输出(改数据库表)，查询则遍历ResultSet打印各行。

```

package chp12_JDBC.exp12_3;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class TransactionTest {

    public static void main(String args[]) {

        String url = "jdbc:mysql://localhost/test";
        Connection con = null;
        Statement stmt;
        PreparedStatement updateSales;
        PreparedStatement updateTotal;

        String updateString = "update COFFEES " +
            "set SALES = ? where COF_NAME = ?";

        String updateStatement = "update COFFEES " +
            "set TOTAL = TOTAL + ? where COF_NAME = ?";
        String query = "select COF_NAME, SALES, TOTAL from COFFEES";

        try {
            Class.forName("com.mysql.jdbc.Driver");
        } catch (java.lang.ClassNotFoundException e) {
            System.err.print("ClassNotFoundException: ");
            System.err.println(e.getMessage());
        }
    }
}

```

```

    }

    try {

        con = DriverManager.getConnection(url, "root", "java");

        //创建两个预编译语句。
        updateSales = con.prepareStatement(updateString);
        updateTotal = con.prepareStatement(updateStatement);
        int [] salesForWeek = {175, 150, 60, 155, 90};
        String [] coffees = {"Colombian", "French_Roast",
                             "Espresso", "Colombian_Decaf",
                             "French_Roast_Decaf"};

        int len = coffees.length;

        //关闭事务自动更新模式。
        con.setAutoCommit(false);

        //循环的每次执行都作为一个单独的事务。
        for (int i = 0; i < len; i++) {
            updateSales.setInt(1, salesForWeek[i]);
            updateSales.setString(2, coffees[i]);
            updateSales.executeUpdate();

            updateTotal.setInt(1, salesForWeek[i]);
            updateTotal.setString(2, coffees[i]);
            updateTotal.executeUpdate();
            con.commit();
        }

        //恢复事务自动更新模式。
        con.setAutoCommit(true);

        updateSales.close();
        updateTotal.close();

        stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery(query);

        while (rs.next()) {
            String c = rs.getString("COF_NAME");
            int s = rs.getInt("SALES");
            int t = rs.getInt("TOTAL");
            System.out.println(c + "      " + s + "      " + t);
        }

        stmt.close();
        con.close();

    } catch (SQLException ex) {
        System.err.println("SQLException: " + ex.getMessage());
        if (con != null) {
            try {
                System.err.println("Transaction is being rolled back");
                con.rollback(); //如果事务执行中有异常抛出，则撤消事务。
            } catch (SQLException excep) {
                System.err.println("SQLException: "+excep.getMessage());
            }
        }
    }
}

```

► **运行结果：**连接数据库执行SQL；增删改无控制台输出(改数据库表)，查询则遍历ResultSet打印各行。

第四部分 SQL 基础操作速查（配例子）

JDBC 里 executeUpdate / executeQuery 执行的就是这些 SQL。下面以一张学生表 Student 为例。增删改用 executeUpdate，查询用 executeQuery。

0. 建表（CREATE TABLE）

```
CREATE TABLE Student (  
    sCode    VARCHAR(20) PRIMARY KEY,    -- 学号,主键  
    sName    VARCHAR(50) NOT NULL,       -- 姓名,非空  
    sAge     INT,                         -- 年龄  
    sMajor   VARCHAR(50)                 -- 专业  
);
```

说明：PRIMARY KEY 主键（唯一、非空）；NOT NULL 该列不能为空；常用类型 INT 整数、VARCHAR(n) 变长字符串、FLOAT 小数、DATE 日期。

1. 插入数据（INSERT）—— executeUpdate

```
-- 指定列插入(推荐)  
INSERT INTO Student(sCode, sName, sAge, sMajor)  
VALUES('20230001', '孟宪喆', 20, '自动化');  
-- 不写列名,必须按建表顺序给全部值  
INSERT INTO Student VALUES('20230002', '徐海洋', 21, '自动化');
```

2. 查询数据（SELECT）—— executeQuery

```
-- 查全部列  
SELECT * FROM Student;  
-- 查指定列  
SELECT sName, sAge FROM Student;  
-- 条件查询 WHERE  
SELECT * FROM Student WHERE sAge > 20;  
SELECT * FROM Student WHERE sMajor = '自动化';  
-- 多条件 AND / OR  
SELECT * FROM Student WHERE sAge >= 20 AND sMajor = '自动化';  
-- 模糊查询 LIKE ( % 匹配任意多字符, _ 匹配一个字符 )  
SELECT * FROM Student WHERE sName LIKE '孟%';    -- 姓孟的  
-- 范围 IN / BETWEEN  
SELECT * FROM Student WHERE sCode IN ('20230001', '20230002');  
SELECT * FROM Student WHERE sAge BETWEEN 18 AND 22;
```

例：查 SELECT sName, sAge FROM Student 的结果

sName	sAge
孟宪喆	20
徐海洋	21

3. 修改数据（UPDATE）—— executeUpdate

```
-- ★一定要带 WHERE,否则改全表!  
UPDATE Student SET sName = '陈嘉宇' WHERE sCode = '20230001';  
-- 同时改多列  
UPDATE Student SET sAge = 22, sMajor = '计算机' WHERE sCode = '20230002';
```

4. 删除数据（DELETE）—— executeUpdate

```
-- ★一定要带 WHERE,否则删全表!  
DELETE FROM Student WHERE sCode = '20230001';  
-- 删除年龄小于18的所有行  
DELETE FROM Student WHERE sAge < 18;
```

5. 排序与统计

```
-- 排序 ORDER BY ( ASC 升序默认 / DESC 降序 )  
SELECT * FROM Student ORDER BY sAge DESC;    -- 按年龄降序  
-- 统计函数: COUNT 计数, AVG 平均, MAX/MIN 最值, SUM 求和  
SELECT COUNT(*) FROM Student;               -- 总人数  
SELECT AVG(sAge) FROM Student;               -- 平均年龄  
SELECT MAX(sAge), MIN(sAge) FROM Student;    -- 最大/最小年龄  
-- 分组统计 GROUP BY: 每个专业多少人  
SELECT sMajor, COUNT(*) FROM Student GROUP BY sMajor;
```

例：SELECT sMajor, COUNT(*) FROM Student GROUP BY sMajor 的结果

sMajor	COUNT
自动化	2
计算机	1

6. SQL 操作速查表

操作	关键字	Java 执行方法	例子
建表	CREATE TABLE	executeUpdate	CREATE TABLE Student(...)
增	INSERT INTO	executeUpdate	INSERT INTO Student VALUES(...)
删	DELETE FROM	executeUpdate	DELETE FROM Student WHERE ...

改	UPDATE...SET	executeUpdate	UPDATE Student SET ... WHERE ...
查	SELECT...FROM	executeQuery	SELECT * FROM Student WHERE ...
条件	WHERE	—	WHERE sAge > 20
模糊	LIKE	—	WHERE sName LIKE '孟%'
排序	ORDER BY	—	ORDER BY sAge DESC
分组	GROUP BY	—	GROUP BY sMajor
统计	COUNT/AVG/MAX/MIN/SUM	—	SELECT COUNT(*) ...

记忆要点：增删改 = executeUpdate （返回受影响行数 int）；查 = executeQuery （返回 ResultSet）。UPDATE 和 DELETE 千万别忘 WHERE，否则操作整张表。